

DATA266 Lab 1 — Team 41 Report

LLM Pretraining • Sentiment Classification • CycleGAN Style Transfer

Course: DATA266, Fall 2026

Team: 41

Members: Yuyao Ding and Pratiksha Kaushik

Submission date: October 6, 2026

Repository: [DATA266 Pair 41 Lab 1](#)

This report presents both members' verified individual results for Tasks 1–3: Yuyao Ding's formal runs, two-rater audit, Kaggle Public score and Mac smoke test, and Pratiksha Kaushik's Task 1–3 runs, two-rater audit and Kaggle Public score. Local FID and MiFID values are not Kaggle leaderboard scores.

Team Ownership Statement

Yuyao Ding is responsible for the implementations, experiments, evaluations, and individual analyses under the Yuyao_Ding folders for Tasks 1–3. Pratiksha Kaushik is responsible for the implementations, experiments, evaluations, and individual analyses under the Pratiksha_Kaushik folders for Tasks 1–3. The joint comparisons were written from both members' recorded results.

Report Contents

1. Experiment overview and evidence
2. Task 1 language model
3. Task 2 sentiment classification
4. Task 3 CycleGAN style transfer
5. Reproducibility and artifact access
6. Team conclusions and references
7. Appendices with complete reviews, figures, and submission checks

1 Experiment Overview and Evidence

1.1 Summary of Reported Models

Task	Yuyao Ding	Pratiksha Kaushik
Task 1 language model	Character-level decoder Transformer; train_20260927T000510Z_b9df2210	Character-level decoder Transformer (8 layers, 8 heads, 512-d, context 512); 20261005-223946_L8H8C512T512
Task 2 baseline	Masked mean pooling with learned embeddings; task2_formal_20260927T020054Z_ba87cd57_baseline	Masked mean pooling with learned 128-d embeddings; all-model run of 2026-10-05
Task 2 experiment 1	TextCNN; task2_formal_20260927T020054Z_ba87cd57_textcnn	CNN, kernels 3/5/7 × 128 channels, global max pooling
Task 2 experiment 2	BiLSTM; task2_formal_20260927T020054Z_ba87cd57_bilstm	BiGRU, 128 hidden units per direction, masked mean pooling
Task 3 CycleGAN	Nine-block CycleGAN; task3_formal_20260927T075834Z_d335ae80	Nine-block CycleGAN with 2-scale PatchGAN discriminators (v3); EMA average of epochs 60/65/70

1.2 Hardware and Software

Member and task	Exact CPU	Exact GPU or device	Python and framework versions	Environment evidence
Yuyao Task 1	Intel Core i9-14900KF	NVIDIA GeForce RTX 4090	Python 3.12.14; PyTorch 2.8.0+cu128	reproducibility/manifests/Yuyao_Ding/train_20260927T000510Z_b9df2210/manifest.json
Yuyao Task 2	Intel Core i9-14900KF	NVIDIA GeForce RTX 4090	Python 3.12.14; PyTorch 2.8.0+cu128	reproducibility/manifests/Yuyao_Ding/task2_formal_20260927T020054Z_ba87cd57/environment.json
Yuyao Task 3	Intel Core i9-14900KF	NVIDIA GeForce RTX 4090	Python 3.12.14; PyTorch 2.8.0+cu128	reproducibility/manifests/Yuyao_Ding/task3_formal_20260927T075834Z_d335ae80/run.json
Pratiksha Task 1	Not recorded (Google Colab host)	NVIDIA A100-SXM4-40GB	Python 3.13.15; PyTorch 2.11.0+cu130; CUDA 13.0; bf16 + torch.compile	reproducibility/manifests/Pratiksha_Kaushik/task1_20261005-223946_L8H8C512T512/
Pratiksha Task 2	Not recorded (Google Colab host)	NVIDIA Tesla T4	PyTorch 2.11.0+cu130 (Colab Python)	reproducibility/manifests/Pratiksha_Kaushik/task2_20261005_all_models/manifest.json
Pratiksha Task 3	Not recorded	NVIDIA GeForce RTX 4090	Python 3.10.13; PyTorch 2.1.2; bf16 autocast	reproducibility/manifests/Pratiksha_Kaushik/task3_20261001_cyclegan_v3/manifest.json

1.3 Evidence and Repository History

Each Yuyao result below is tied to its run ID, configuration, checkpoint, metric record, and original logs.

Source and audit records are in reproducibility/manifests/Yuyao_Ding and reproducibility/raw_logs/Yuyao_Ding. The three individual reports in report/ provide the detailed provenance and scope. Pratiksha's results are tied in the same way to reproducibility/manifests/Pratiksha_Kaushik and reproducibility/raw_logs/Pratiksha_Kaushik; her three task reports are in report/Pratiksha_Kaushik/.

Yuyao completed the formal experiments on a separate Windows 11 computer with an NVIDIA RTX 4090, then transferred the recorded outputs to the shared repository in one batch. This explains the single large update. The team comparison and final submission were not completed by that transfer.

Artifact access

- Task-specific restoration links and destinations:
https://github.com/Ashley96yy/DATA266_Pair41_Lab1/blob/main/README.md (Artifact access).
- Pratiksha's processed data and checkpoints: Google Drive links for each task are in the repository README (section "Pratiksha's work and artifact downloads") and in each task's Pratiksha_Kaushik/README.md and checkpoints/README.md.
- TinyStories: task1_llm/data/; Yelp Polarity parquet files: task2_sentiment/data/; Monet and photo JPEGs: task3_gan/data/monet_jpg/ and photo_jpg/. Source download links and exact layouts:
https://github.com/Ashley96yy/DATA266_Pair41_Lab1/blob/main/README.md#raw-datasets.

2 Task 1 Language Model from Scratch

2.1 Data and Evaluation Protocol

Item	Yuyao Ding	Pratiksha Kaushik
Dataset version and file identification	Original TinyStories train and validation text files; see preprocessing manifest for file hashes.	roneneldan/TinyStories (Hugging Face), train partition (2,119,719 stories); text repaired with ftfy and curly quotes/dashes mapped to ASCII.
Training and validation sizes and units	100,000 training stories; 10,000 validation stories.	100,000 training stories; 10,000 validation stories (89,656,644 / 8,991,471 characters).
Sampling seeds and split procedure	Training seed 640; validation seed 641; official splits preserved.	Seed 1337; seeded shuffle of the train partition, stories under 50 characters dropped, duplicates removed, then split. Indices in data_processed/split_indices.json.
Character vocabulary size	104 characters, including UNK.	80 symbols: 78 characters seen at least 20 times in training, plus <eos> and <unk>.
Unknown-character handling	UNK for characters outside training vocabulary.	<unk> for characters outside the training vocabulary (rate 0.00013% train, 0.00021% validation).
Context length and sequence construction	256-character input windows; next-character targets shifted by one; stride 256.	512-character input windows; next-character targets shifted by one; non-overlapping windows with a random offset each epoch (175,109 windows per epoch).
Overlap or leakage checks	Vocabulary fit on training text only; selected training-story hashes excluded from validation.	Vocabulary fit on training text only; stories deduplicated before splitting, so no story is in both splits.
Evaluation checkpoint	Run train_20260927T000510Z_b9df2210, epoch 10, epoch_010.pt.	Run 20261005-223946_L8H8C512T512, epoch 10 (step 27,360), ckpt_best.pt (best = last).
Generation settings	Three prompts; 300 new characters each; greedy and temperature 0.8; seeds 640–642.	Four prompts (e.g. “Once upon a time”); greedy, T=0.5, T=0.8, T=1.0 with top-k 20, T=1.3; about 530 characters per sample; unconditional samples at T=0.8.

Yuyao used the official TinyStories training and validation files, with separate seeded story samples. Pratiksha drew both of her splits from the TinyStories train partition with seed 1337 and removed duplicates before splitting. Each member’s training and validation metrics use the same selected checkpoint. Because the splits, vocabularies (104 vs 80 symbols) and context lengths (256 vs 512) differ, the two members’ numbers are not a controlled comparison.

2.2 Architecture and Hyperparameters

Setting	Yuyao Ding	Pratiksha Kaushik
Transformer blocks	2	8

Setting	Yuyao Ding	Pratiksha Kaushik
Attention heads	4	8
Embedding dimension	128	512
Feed-forward dimension	512	2,048 (512 → 2,048 → 512, GELU)
Token and positional embeddings	Learned token and position embeddings.	Learned token embeddings (80 × 512) and learned absolute position embeddings (512 × 512); untied output head.
Layer normalization and residual design	Pre-normalized LayerNorm (epsilon 1e-5) with residual attention and feed-forward paths.	Pre-LayerNorm blocks with residual attention and feed-forward paths, plus a final LayerNorm; residual output projections initialised with std 0.02/√16.
Causal masking	Manually implemented causal self-attention mask; source and smoke checks in Task 1 repository.	Manually built lower-triangular causal mask; tested so that attention weights above the diagonal are zero and editing future tokens leaves earlier logits unchanged.
Dropout	0.1	0.1
Optimizer and weight decay	AdamW; betas 0.9/0.95; weight decay 0.01.	AdamW; betas 0.9/0.95; weight decay 0.1 on weight matrices only.
Learning rate, warm-up, and schedule	2,710-step linear warm-up to 3e-4; cosine decay to 3e-5.	820-step linear warm-up to 8e-4; cosine decay to 8e-5.
Batch size	64	64 sequences (32,768 characters per step)
Epochs and optimizer steps	10 epochs; 54,200 optimizer steps.	10 epochs; 27,360 optimizer steps.
Gradient clipping and precision	Global L2 clip at 1.0; float32.	Global L2 clip at 1.0; bf16 autocast + torch.compile (softmax and loss in fp32).
Parameter count	456,192	25,564,160

Independent design differences:

Yuyao manually implemented causal multi-head attention, pre-normalized blocks, GELU feed-forward layers, residual connections, and learned token/position embeddings. Her source does not use a prebuilt Transformer or attention module. Pratiksha also implemented multi-head self-attention (fused QKV projection, fp32 softmax), the causal mask, LayerNorm, the GELU feed-forward network, residual connections, token and position embeddings and the language-model head from tensor operations, without `nn.Transformer`, `nn.MultiheadAttention` or `scaled_dot_product_attention`.

2.3 Evaluation Results

Metric	Yuyao Ding	Pratiksha Kaushik
Training cross-entropy loss	0.882596 nats/character (full selected training split)	0.447569 nats/character (training split)
Validation cross-entropy loss	0.882114 nats/character (full selected validation split)	0.474203 nats/character (full validation split)

Metric	Yuyao Ding	Pratiksha Kaushik
Perplexity and evaluated split	Train 2.417166; validation 2.416001	Train 1.564504; validation 1.606734
Bits per character and evaluated split	Train 1.273317; validation 1.272621	Train 0.645705; validation 0.684131
Generalization gap: validation minus training loss	-0.000482 nats/character (validation minus training; epoch-10 checkpoint)	+0.026635 nats/character (validation minus training; +5.95%; epoch-10 checkpoint)
Top-1 next-character accuracy	Train 72.2687%; validation 72.3116%	Train 85.4367%; validation 84.7353%
Distinct-1	Greedy 0.038889; T=0.8 0.045556	Greedy 0.125388; T=0.8 0.211786
Distinct-2	Greedy 0.189521; T=0.8 0.269788	Greedy 0.378849; T=0.8 0.654618
Distinct-3	Greedy 0.300895; T=0.8 0.564877	Greedy 0.504270; T=0.8 0.868764
Repeated 4-gram rate	Greedy 0.630752; T=0.8 0.267116	Greedy 0.059204; T=0.8 0.013522 (real validation stories 0.010912)
Gradient norm, summary, and clipping stage	Pre-clipping L2 mean 0.715206; median 0.637479; max 4.003296; 3,922 steps >1	Pre-clipping L2 after warm-up: mean 0.154129; median 0.144855; p99 0.340872; max over all steps 25.532104; 1.06% of steps clipped at 1.0
Stability: loss spikes, NaN and Inf events	0 loss spikes; 0 nonfinite loss/gradient steps	0 loss spikes (loss > 1.3 × moving average after warm-up); 0 non-finite loss/gradient steps
Parameter count	456,192	25,564,160
Training tokens per second	746,664.1530 target characters/s (training loop)	452,013.26 target characters/s (steady-state optimizer steps)
Generation tokens per second	Greedy 271.9974; T=0.8 247.3769 generated characters/s	96.05 characters/s single stream; 2,182.48 characters/s at batch 32 (no KV-cache)
Peak memory, measurement type, and unit	CUDA allocated peak 750,842,880 bytes; process RSS peak 2,362,290,176 bytes	CUDA allocated peak 10.587 GiB; reserved peak 10.789 GiB
Total training time	Training loop 1,189.194 s; full train/eval wall 1,247.819 s	36.91 min (2,214.6 s) for 10 epochs including evaluation

Cross-entropy is token-weighted over the full selected split at the same epoch-10 checkpoint, with dropout off. Perplexity = $\exp(\text{CE})$; bits/character = $\text{CE}/\ln(2)$. Distinct-n is pooled unique/total generated n-grams, excluding prompts; repeated-4 is the duplicate fraction. Training throughput counts target characters and excludes evaluation; full wall includes evaluation/checkpointing but not preprocessing or generation. Generation timing had no separate warm-up. CUDA peak is allocated tensors; RSS is a process-lifetime host-memory peak. Pratiksha's distinct-n and repeated-4-gram rates are measured on an equal budget of 2,257 generated words per setting, so they are not directly comparable with Yuyao's budget.

Figure 1 Yuyao training and validation loss curves

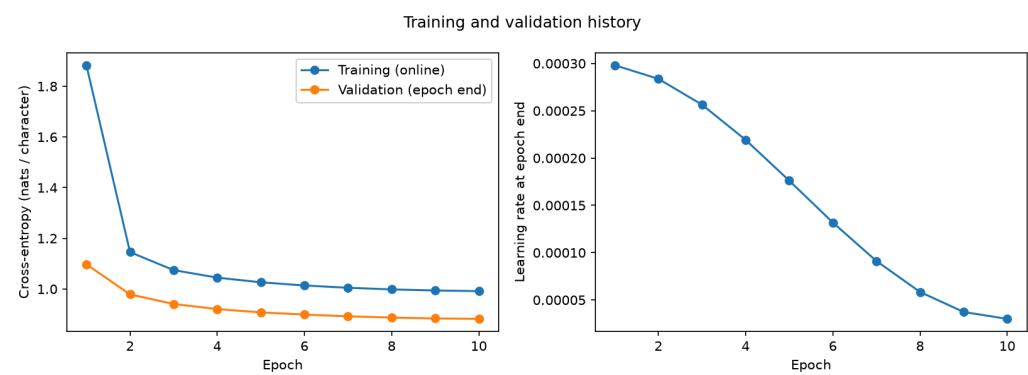
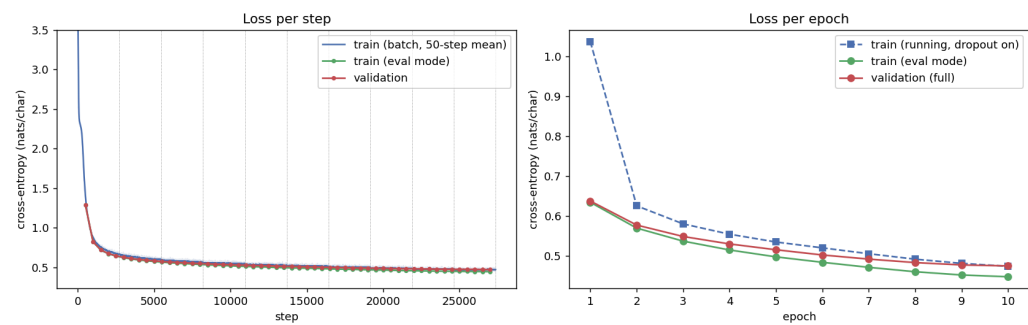


Figure 2 Pratiksha training and validation loss curves



2.4 Generated Samples and Failure Analysis

Three observed failure cases are reported for each member, with the exact generated snippet, the prompt and the decoding settings.

Member and case	Prompt and actual generated text	Failure type	Observation and possible explanation
Yuyao 1	Prompt: "The dog wanted to"; greedy. "They were happy and sad. They had a big box of the box. They were happy and sad. They had a big box of the box."	Repetition	Repeated phrases do not advance the story.
Yuyao 2	Prompt: "A little girl found"; T=0.8, seed 641. "It's okay and you can buy it not mean to buy like to play with my friend."	Broken grammar	Verb phrases are joined without a clear relationship.
Yuyao 3	Prompt: "Once upon a time,"; T=0.8, seed 640. "The bird loved to take a special music. They were happy and loved to play together. One day, Lily's mom was delicious and called Ben."	Loss of coherence	Unintroduced bird and implausible description of Lily's mother.
Pratiksha 1	Prompt: "The sun was hot and the bird"; greedy. "The birds were singing and the sun was shining. The birds were singing and the sun was shining. The birds were singing and the sun was shining."	Repetition	Repeated 4-gram rate 86.6% in this sample (1.1% in real stories). Greedy decoding keeps choosing the same high-probability phrase, so the context repeats and the model loops.
Pratiksha 2	Prompt: "The sun was hot and the bird"; T=1.3. "Peter was svery exhausted, but they decided to keep going. It seemed like someday it fit up heavily in her green home."	Broken grammar and invented words	Invented-word rate 4.0% (0.03% at T=0.8): "svery", "originalt", "aira". High temperature samples unlikely characters inside words, and each error makes the next one more likely.
Pratiksha 3	Prompt: "Tom and his dog went to the park."; T=0.8, 1,533 characters. Opening: "They saw many things: trees, flowers, birds, dogs." Ending: "And you have to say sorry to the sandbox. I will keep the sand away."	Loss of coherence	Only 7.4% of content words are shared between the two halves and the opening characters disappear. The 512-character context covers about 100 words, so the start of the story falls out of view.

2.5 Joint Analysis

Observed strengths and weaknesses: Pratiksha's 25.6M-parameter model reaches validation CE 0.4742 (0.684 bits/character, 84.7% top-1) against 0.8821 (1.273 bits/character, 72.3%) for Yuyao's 0.46M-parameter model, and its T=0.8 samples are more diverse (distinct-3 0.869 vs 0.565). Yuyao's model is about 56× smaller, needs far less memory (0.75 GB vs 10.6 GiB) and generates faster per stream (272 vs 96 characters/s). Both runs were stable, with no loss spikes or non-finite steps.

Interpretation and limitations: The two runs differ in model size, context length (256 vs 512), vocabulary, data split and hardware at the same time, so the quality gap reflects these changes together rather than one design choice. Yuyao's near-zero generalization gap points to an under-fitted small model; Pratiksha's +0.027 gap is still small. Both models share the same failure types: repetition under greedy decoding and loss of coherence over long stories.

Next experiment: Train one size sweep (for example 2L/128d, 4L/256d, 8L/512d) on a single shared split with a fixed context, to separate the effect of capacity from context length, and add a KV-cache to remove the generation-speed penalty of the larger model.

Yuyao evidence: task1_llm/Yuyao_Ding/metrics_report.csv; task1_llm/Yuyao_Ding/failure_analysis.md; reproducibility/raw_logs/Yuyao_Ding/train_20260927T000510Z_b9df2210.jsonl.

Pratiksha evidence: task1_llm/Pratiksha_Kaushik/metrics_report.csv;
task1_llm/Pratiksha_Kaushik/failure_analysis.md;
reproducibility/raw_logs/Pratiksha_Kaushik/task1_pratiksha_kaushik_20261005-223946_L8H8C512T512_run
log_from_notebook_output.txt.

3 Task 2 Sentiment Classification

3.1 Data Preparation and Evaluation Comparability

Item	Yuyao Ding	Pratiksha Kaushik
Dataset and source version	Yelp Polarity original parquet training and official test files; 560,000/38,000 source rows.	Yelp Polarity, fancyzhx/yelp_polarity (Hugging Face); 560,000/38,000 source rows.
Training, validation, and test counts	After audit: train 503,934; validation 55,993; official test 38,000.	Stratified samples: train 18,000; validation 2,000 (from 20,000 training rows); test 5,000 (from the official test set).
Class distribution	Train 251,959 negative/251,975 positive; validation 27,996/27,997; test 19,000/19,000.	Exactly balanced in all splits (test 2,500 negative / 2,500 positive).
Review-length distribution	Training-derived raw-word quartiles at 51 and 173 words; 1,751 test reviews exceed 192 processed tokens.	Source training reviews: mean 133 words, median 97, 95th percentile 372. Slices use processed tokens: short ≤ 40 , medium 41–100, long > 100 .
Missing or malformed entries	No invalid training rows; empty after preprocessing: 35 train, 4 validation, 1 test; empty inputs use UNK.	No missing text or labels; rows with invalid labels or empty processed text are dropped (all sampled splits came out full size).
Duplicate and overlap handling	Removed 4 conflicting-label rows, 62 duplicate rows, 7 train/test text overlaps; identity uses lowercased normalized whitespace.	No explicit duplicate or train/test overlap removal (listed as a limitation).
Lowercasing and punctuation handling	Lowercase; normalize selected contractions and curly apostrophes; remove punctuation.	Lowercase; every character except a–z, 0–9, whitespace and apostrophes replaced by a space. Literal “\n” sequences leave a stray “n” token (limitation).
Stopwords, stemming, and lemmatization	Stopwords removed with configured negation/contrast exceptions; no stemming or lemmatization.	47-word stopwords list removed; not/never/no kept; “but” removed (limitation). No stemming or lemmatization.
Tokenization and vocabulary size	Word tokenizer; up to 50,000 entries including special tokens; minimum training frequency 2.	Word regex tokenizer; 30,000 most frequent training tokens including <pad> and <unk>.
Maximum sequence length	First 192 processed tokens, then PAD; empty input receives UNK.	First 180 processed tokens; padded per batch to the longest sequence (minimum 7).
Embedding initialization	Trainable 128-dimensional embeddings; random initialization; no pretrained embeddings.	Trainable random embeddings (128-d baseline, 160-d CNN and BiGRU); no pretrained embeddings.

Item	Yuyao Ding	Pratiksha Kaushik
Split seed and decision threshold	Stratified split seed 640; fixed positive probability threshold 0.5.	Seed 9002 for sampling and split; per-model threshold tuned on validation macro-F1 over 0.20–0.80: 0.53 / 0.56 / 0.27.

Yuyao's three models use identical cleaned training/validation splits and the same 38,000 official test reviews. Pratiksha's three models share one stratified sample (18,000 train, 2,000 validation, 5,000 test; seed 9002) of the same Yelp Polarity source. Each member's models are therefore comparable with each other, but training size (about 504,000 vs 18,000 reviews), test set (38,000 vs 5,000) and threshold rule (fixed 0.5 vs validation-tuned) differ between members, so a six-model controlled comparison is not possible.

3.2 Model Lineup and Design Choices

Model IDs: Y-B = Yuyao baseline; Y-E1 = Yuyao experiment 1; Y-E2 = Yuyao experiment 2; P-B = Pratiksha baseline; P-E1 = Pratiksha experiment 1; P-E2 = Pratiksha experiment 2.

ID	Architecture	Embedding and hidden dimensions	Main training hyperparameters	Selected epoch	Design rationale
Y-B	Mean pooling	128-d embedding; mean pooled	Adam 1e-3; batch 256; dropout 0.30; seed 640	5	Simple trainable baseline.
Y-E1	TextCNN	128-d embedding; 128 filters each at widths 3/4/5	Adam 1e-3; batch 256; dropout 0.50; seed 640	3	Tests local n-gram feature extraction.
Y-E2	BiLSTM	128-d embedding; 128 hidden units/direction	Adam 1e-3; batch 256; dropout 0.40; clip 1; seed 640	3	Tests bidirectional sequential context.
P-B	Mean pooling	128-d embedding; masked mean pooled	AdamW 1e-3, weight decay 1e-4; batch 128; dropout 0.20; clip 1.0; seed 9002	5 (last)	Simple trainable baseline.
P-E1	CNN	160-d embedding; 128 channels each at kernel widths 3/5/7; global max pooling	AdamW 1e-3, weight decay 1e-4; batch 128; dropout 0.35; clip 1.0; seed 9002	5 (last)	Tests local phrase features such as “not good” and “least favorite”.
P-E2	BiGRU	160-d embedding; 128 hidden units/direction; masked mean pooling	AdamW 1e-3, weight decay 1e-4; batch 128; dropout 0.35; clip 1.0; seed 9002	5 (last)	Tests bidirectional sequential context.

Cross-member differences:

Yuyao’s three models and their embeddings were trained from scratch. Pratiksha’s three models and their embeddings were also trained from scratch from random initialization. No pretrained embeddings or language models were used by either member.

3.3 Classification and Calibration Results

Metric	Y-B	Y-E1	Y-E2	P-B	P-E1	P-E2
Accuracy	0.932447	0.938842	0.947526	0.891400	0.905400	0.888000
Precision macro	0.932461	0.938904	0.947553	0.892092	0.905405	0.888340
Recall macro	0.932447	0.938842	0.947526	0.891400	0.905400	0.888000
F1 macro	0.932447	0.938840	0.947526	0.891352	0.905400	0.887975
Precision micro	0.932447	0.938842	0.947526	0.891400	0.905400	0.888000
Recall micro	0.932447	0.938842	0.947526	0.891400	0.905400	0.888000

Metric	Y-B	Y-E1	Y-E2	P-B	P-E1	P-E2
F1 micro	0.932447	0.938842	0.947526	0.891400	0.905400	0.888000
Precision weighted	0.932461	0.938904	0.947553	0.892092	0.905405	0.888340
Recall weighted	0.932447	0.938842	0.947526	0.891400	0.905400	0.888000
F1 weighted	0.932447	0.938840	0.947526	0.891352	0.905400	0.887975
ROC-AUC	0.979742	0.984943	0.988794	0.957359	0.965983	0.960139
PR-AUC	0.979582	0.985341	0.989155	0.957428	0.965196	0.960752
Matthews correlation coefficient	0.864908	0.877746	0.895079	0.783491	0.810805	0.776340
Brier score	0.050910	0.045303	0.039617	0.082544	0.071710	0.090401
Expected calibration error	0.005033	0.002623	0.009108	0.052954	0.030130	0.071114

Positive label = 1; decision threshold = 0.5. Macro averages classes equally, weighted averages by support, and micro aggregates counts. PR-AUC is trapezoidal area; average precision is a distinct metric (reported in Yuyao's individual results). Brier is mean squared probability error; ECE uses ten equal-width predicted-class confidence bins. Pratiksha's columns use her 5,000-review test sample and per-model thresholds tuned on validation (P-B 0.53, P-E1 0.56, P-E2 0.27); her PR-AUC is scikit-learn average precision.

3.4 Confusion Matrices and Statistical Comparisons

Model	True negative	False positive	False negative	True positive
Y-B	17769	1231	1336	17664
Y-E1	17725	1275	1049	17951
Y-E2	18076	924	1070	17930
P-B	2281	219	324	2176
P-E1	2259	241	232	2268
P-E2	2183	317	243	2257

Numeric confusion matrices for both members are in the table above. Yuyao's labeled plots and evaluation JSON are under `task2_sentiment/Yuyao_Ding/outputs/task2_formal_20260927T020054Z_ba87cd57/{baseline,textcnn,bilstm}/`. Pratiksha's plots are `task2_sentiment/Pratiksha_Kaushik/outputs/confusion_matrix_*.png`.

Model	Accuracy 95% bootstrap CI	Macro-F1 95% bootstrap CI	MCC 95% bootstrap CI
Y-B	0.929947–0.935159	0.929946–0.935156	0.859910–0.870349
Y-E1	0.936474–0.941369	0.936473–0.941360	0.873040–0.882777
Y-E2	0.945474–0.949921	0.945472–0.949918	0.890981–0.899865
P-B	0.882600–0.899600	0.882596–0.899563	0.766233–0.799679
P-E1	0.897400–0.913605	0.897396–0.913605	0.794952–0.827303
P-E2	0.878995–0.896600	0.878800–0.896460	0.758357–0.793489

1,000 paired bootstrap resamples of test rows, seed 640; 2.5th/97.5th percentile bounds. Intervals capture test-row sampling uncertainty, not variation from retraining or business-level clustering. Pratiksha's intervals use 1,000 bootstrap resamples of her 5,000 test rows with seed 9002.

Paired McNemar comparison	Baseline correct and experiment wrong	Baseline wrong and experiment correct	p-value	Interpretation
Y-B versus Y-E1	770	1,013	9.484×10^{-9} (Holm-adjusted)	TextCNN exceeded Yuyao's baseline on the shared test set (adjusted $p < 0.05$).
Y-B versus Y-E2	669	1,242	2.137×10^{-39} (Holm-adjusted)	BiLSTM exceeded Yuyao's baseline on the shared test set (adjusted $p < 0.05$).
P-B versus P-E1	168	238	5.991×10^{-4} (Holm-adjusted 1.198×10^{-3})	CNN exceeded Pratiksha's baseline on her test sample (adjusted $p < 0.05$).
P-B versus P-E2	275	258	0.4883 (Holm-adjusted 0.4883)	No significant difference between BiGRU and Pratiksha's baseline.

Exact paired McNemar test on the same test IDs; familywise correction by Holm for two baseline comparisons, alpha 0.05. Both experimental models exceed Yuyao's baseline in this one run. No TextCNN-versus-BiLSTM significance claim is made. For Pratiksha, only the CNN improvement is significant; the BiGRU is not distinguishable from her baseline.

3.5 Slice Robustness and Resource Use

Rows are given for every reported model and slice. Slices are defined without using test outcomes.

Model	Slice definition	Test examples	Macro-F1	Error rate
Y-B	length_short	9590	0.9294	0.0678
Y-E1	length_short	9590	0.9346	0.0626

Model	Slice definition	Test examples	Macro-F1	Error rate
Y-E2	length_short	9590	0.9457	0.0521
Y-B	length_medium	18952	0.9338	0.0662
Y-E1	length_medium	18952	0.9415	0.0585
Y-E2	length_medium	18952	0.9493	0.0507
Y-B	length_long	9458	0.9262	0.0700
Y-E1	length_long	9458	0.9318	0.0650
Y-E2	length_long	9458	0.9406	0.0564
Y-B	has_negation	27949	0.9253	0.0721
Y-E1	has_negation	27949	0.9350	0.0629
Y-E2	has_negation	27949	0.9444	0.0537
Y-B	no_negation	10051	0.9260	0.0549
Y-E1	no_negation	10051	0.9236	0.0562
Y-E2	no_negation	10051	0.9342	0.0491
Y-B	truncated	1751	0.9038	0.0862
Y-E1	truncated	1751	0.9030	0.0879
Y-E2	truncated	1751	0.9057	0.0845
Y-B	not_truncated	36249	0.9333	0.0667
Y-E1	not_truncated	36249	0.9401	0.0599
Y-E2	not_truncated	36249	0.9491	0.0509
Y-B	empty_after_preprocessing	1	0	1.0000
Y-E1	empty_after_preprocessing	1	0	1.0000
Y-E2	empty_after_preprocessing	1	0	1.0000
P-B	length_short (≤ 40 tokens)	1622	0.8844	0.1122
P-E1	length_short (≤ 40 tokens)	1622	0.9000	0.0968
P-E2	length_short (≤ 40 tokens)	1622	0.8720	0.1208
P-B	length_medium (41–100)	1921	0.8868	0.1130
P-E1	length_medium (41–100)	1921	0.9053	0.0947
P-E2	length_medium (41–100)	1921	0.8953	0.1046
P-B	length_long (> 100)	1457	0.8968	0.0988

Model	Slice definition	Test examples	Macro-F1	Error rate
P-E1	length_long (> 100)	1457	0.9048	0.0920
P-E2	length_long (> 100)	1457	0.8838	0.1119
P-B	has_negation	2956	0.8765	0.1123
P-E1	has_negation	2956	0.8965	0.0961
P-E2	has_negation	2956	0.8734	0.1177
P-B	no_negation	2044	0.8789	0.1032
P-E1	no_negation	2044	0.8908	0.0925
P-E2	no_negation	2044	0.8742	0.1037
P-B	high_punctuation (≥ 3 ! or ?)	1104	0.9221	0.0779
P-E1	high_punctuation (≥ 3 ! or ?)	1104	0.9401	0.0598
P-E2	high_punctuation (≥ 3 ! or ?)	1104	0.9192	0.0806
P-B	low_punctuation	3896	0.8826	0.1173
P-E1	low_punctuation	3896	0.8955	0.1045
P-E2	low_punctuation	3896	0.8791	0.1209

Length uses training raw-word quartiles (51 and 173). Negation is identified by the predefined word rule; truncated means more than 192 processed tokens. Slices overlap and differ in class balance. Their thresholds and definitions do not use test outcomes. Pratiksha's length slices use processed-token counts (≤ 40 , 41–100, > 100), negation uses the words not/never/no/n't, and high punctuation means three or more "!" or "?". These definitions were fixed before evaluation.

Model	Parameters	Training time	Training examples per second	Peak memory and measurement type
Y-B	6400129	114.32 s	23,888.54/s	166.98 MiB peak CUDA allocation
Y-E1	6597377	138.06 s	19,720.45/s	298.79 MiB peak CUDA allocation
Y-E2	6664449	609.81 s	4,362.04/s	378.29 MiB peak CUDA allocation
P-B	3840129	6.35 s	14,180.67/s	100.47 MiB peak CUDA allocation
P-E1	5107969	18.98 s	4,741.80/s	236.12 MiB peak CUDA allocation
P-E2	5022977	16.03 s	5,615.92/s	558.25 MiB peak CUDA allocation

Yuyao learning curves: baseline, TextCNN, and BiLSTM loss plots from the formal run are reproduced in Appendix B; source files are under task2_sentiment/Yuyao_Ding/outputs/task2_formal_20260927T020054Z_ba87cd57/. Pratiksha's training-loss and validation macro-F1 curves for all three models are in task2_sentiment/Pratiksha_Kaushik/outputs/training_loss_and_val_f1.png (Appendix B).

3.6 Individual Error Reviews

For each member, include the required 20-case review: five confident false positives, five confident false negatives, five near-threshold errors, and five slice-specific failures. State which model or models were reviewed; where reviews were completed separately per model, include each set.

Member	Reviewed models	Number of reviewed cases	Full review location
Yuyao	Baseline, TextCNN, BiLSTM	20 per model; 60 entries, 53 unique reviews	Appendix A and task2_sentiment/Yuyao_Ding/failure_analysis.md
Pratiksha	CNN (P-E1, best model)	20 cases (5 per category)	Appendix A and task2_sentiment/Pratiksha_Kaushik/manual_error_review.md

Each reviewed case is recorded in the format below; the complete reviews are in Appendix A.

Case ID and category	Actual review snippet	True and predicted labels and probability	Error type	Proposed testable fix
test_17640; confident false positive	"my least favorite" → processed "buffets vegas favorite"	Negative → positive; P(positive)=1	Negative comparison word removed	Preserve "least" and compare validation errors on comparative phrases; keep official test fixed.

Observed patterns include comparison/negation loss during preprocessing, target and time shifts in mixed reviews, positive conclusions beyond the 192-token cutoff, and ambiguous short official labels. Model-mechanism explanations and proposed fixes are hypotheses; the 60 reviewed errors are selected cases, not prevalence estimates. Pratiksha's 20 CNN errors show the same themes: mixed sentiment where one strong phrase wins (5), hedged reviews near the threshold (3), a verdict that comes only at the end (2, one cut by her 180-token limit), misread negation (2), sarcasm (2), and single cases of a superlative negative, a removed "but", a competitor as the target, a French review, topic-word bias and possible label noise.

3.7 Joint Analysis

Within Yuyao's one-run comparison, BiLSTM has the highest test accuracy and macro-F1 (0.947526), followed by TextCNN (0.938842) and the mean-pooling baseline (0.932447). BiLSTM takes 609.81 s versus 114.32 s for baseline; TextCNN has the lowest ECE (0.002623). Both experiments improved over baseline under paired McNemar tests with Holm adjustment. This comparison includes different dropout/clipping settings and is not an architecture-only ablation.

Between-member comparison: Yuyao's models score 3–6 points higher (best 0.9475 vs 0.9054 accuracy), mainly because they were trained on about 28× more reviews and use early stopping. In both setups the CNN beats the mean-pooling baseline significantly. The recurrent model ranks in opposite order: Yuyao's BiLSTM is her best model, while Pratiksha's BiGRU is her weakest, because it overfits after epoch 3 on 18,000 reviews and the last epoch was kept. Yuyao's models are also better calibrated (ECE 0.003–0.009 vs 0.030–0.071).

For Yuyao's evidence: one seed, shared 38,000 test rows, and model-specific dropout/clipping changes limit causal interpretation. Long and truncated reviews are weaker. The two members used different test sets and preprocessing, so cross-member differences are descriptive. For Pratiksha's evidence: one seed, a 5,000-review test sample (wider intervals), no early stopping, and preprocessing choices (removed "but", 180-token cutoff) limit interpretation.

Proposed Yuyao follow-up: preserve comparison words and literal escape boundaries in a separately versioned preprocessing pipeline; tune on validation only, then assess the frozen change on the unchanged test set. This experiment has not been run. Proposed Pratiksha follow-up: keep “but”, raise the length limit or use head-plus-tail tokens, add early stopping, and retrain on a much larger sample, changing one factor at a time and comparing on validation only. These experiments have not been run.

task2_sentiment/Yuyao_Ding/metrics_report.csv; task2_sentiment/Yuyao_Ding/failure_analysis.md;
reproducibility/manifests/Yuyao_Ding/task2_formal_20260927T020054Z_ba87cd57/.

task2_sentiment/Pratiksha_Kaushik/metrics_report.csv; task2_sentiment/Pratiksha_Kaushik/metrics.md;
task2_sentiment/Pratiksha_Kaushik/manual_error_review.md; reproducibility/manifests/Pratiksha_Kaushik/task2_20261005_all_models/.

4 Task 3 CycleGAN Image Style Transfer

4.1 Data and Evaluation Protocol

Domain A: Monet paintings. **Domain B:** Photographs. **A to B:** Monet to photo. **B to A:** Photo to Monet.

Item	Yuyao Ding	Pratiksha Kaushik
Dataset source and file identification	Original 300 Monet JPEGs and 7,038 photo JPEGs; task3_data_inventory.json records hashes.	Kaggle “I’m Something of a Painter Myself” data: 300 Monet JPEGs and 7,038 photo JPEGs.
Training, validation, and test counts per domain	Monet train/val/test = 240/30/30; photo = 5,631/703/704. Held-out evaluation uses 30 each.	Monet: all 300 used for training and as the metric reference. Photos: 5,772 train and 1,000 held out; the 300 photos scored by the course script are excluded from training.
Split seed and duplicate handling	Seed 41; 9 duplicate-photo groups assigned without crossing partitions; originals retained.	Seed 9002; no duplicate-image check recorded.
Training image preprocessing	Resize 286×286; random crop 256×256; horizontal flip; normalize to $[-1,1]$.	Resize to 286 and random-crop 256 × 256; DiffAugment (colour + translation) on discriminator inputs.
Checkpoint-selection criterion	Lowest mean bidirectional validation KID; fixed 30 images/domain; 10 subsets of 20.	Course metric (FID + MiFID)/2 in both directions on a separate 300-photo validation set every 5 epochs; final weights are the EMA average of the plateau epochs 60, 65 and 70.
Selected run and epoch	Run task3_formal_20260927T075834Z_d335ae80; epoch 70 best.pt.	v3 final run (80 epochs, 64,000 steps); EMA average of epochs 60/65/70 (checkpoints/best_ema_v3_final.pt, validation score 48.285) plus Monet → photo colour calibration (checkpoints/color_cal.pt).
Evaluation script and version or hash	Course Part3_Evaluation_Script.ipynb definitions, with obsolete scipy sqrtm disp argument removed; implementation notes in Task 3 README.	Course Part3_Evaluation_Script.ipynb definitions copied into the notebook and into evaluate_local.py.
Evaluation images per direction	Held-out 30/direction; separate course protocol 300/direction.	Held-out: 300 Monets → photo and 1,000 held-out photos → Monet; course protocol 300 per direction.
Filename ordering and image-selection rule	First 30 sorted held-out test filenames; first 300 sorted raw filenames for course protocol.	Course protocol: first 300 files by sorted name; held-out photos chosen by seed.
Evaluation batch size	16 for evaluation feature batches; translation export is imagewise.	100 for Inception features; 32 for translation; 16 for the throughput benchmark.

Item	Yuyao Ding	Pratiksha Kaushik
Metric feature extractor and preprocessing	Course Inception-v3 preprocessing for FID/KID/MiFID; AlexNet v0.1 LPIPS; five-nearest-neighbor precision/recall and density/coverage.	torch-fidelity inception-v3-compat (2048-d) for FID/KID/precision-recall/density-coverage/content cosine; torchvision Inception-v3 for the course FID and MiFID; AlexNet LPIPS; k = 5 nearest neighbours.

Yuyao reports two separate protocols. Held-out evaluation uses 30 fixed test images per direction. The course protocol uses the first 300 sorted raw filenames per domain, including training images; it is not a generalization estimate. The supplied evaluation notebook's metric definitions were retained except a SciPy compatibility update removing `sqrtm(dis=False)`. Pratiksha's evaluator and image lists must be verified before direct comparison. Pratiksha also reports two protocols: a held-out evaluation (1,000 photos never used for training against the 300 Monets) and the course protocol on the first 300 sorted files per direction. Her course-protocol photos were excluded from training, so that score has no photo leakage, but the Monet side still contains training paintings.

4.2 Architecture and Training Comparison

Setting	Yuyao Ding	Pratiksha Kaushik
Two-generator architecture	Two nine-residual-block ResNet generators, Monet $\leftrightarrow$ photo; 64 base channels.	Two ResNet generators (photo $\rightarrow$ Monet and Monet $\rightarrow$ photo), 11,378,179 parameters each.
Two-discriminator architecture	Two 70 $\times$ 70 PatchGAN discriminators; 64 base channels.	Two PatchGAN discriminators, each run at 2 scales (full and half resolution); 64 base channels; 5,529,474 parameters each.
Residual blocks and channel widths	9 residual blocks per generator; 64 base channels.	9 residual blocks per generator; 64 base channels.
Normalization and upsampling	InstanceNorm without affine/running statistics; two transpose-convolution upsampling layers.	InstanceNorm; two transpose-convolution upsampling layers; Tanh output.
Adversarial loss	Least-squares adversarial objective in both directions.	Least-squares adversarial objective in both directions, averaged over the 2 discriminator scales.
Cycle-consistency loss and weight	L1(Monet $\rightarrow$ photo $\rightarrow$ Monet)+L1(photo $\rightarrow$ Monet $\rightarrow$ photo), coefficient 10.	L1 in both directions, coefficient 10.
Identity loss and weight	Identity L1 in both domains, coefficient 5.	Identity L1 in both domains, coefficient 1 (the v1 baseline used 5).
Optimizer and learning-rate schedule	Adam $2e-4$, betas 0.5/0.999; 50 constant-rate then 50 linear-decay epochs.	Adam $2e-4$, betas 0.5/0.999; constant for 40 epochs, then linear decay to 0 over 40 epochs.
Batch size and replay pool	Batch 1; replay pool of 50 per domain.	Batch 4; replay pool of 50 per domain; EMA generators (decay 0.999).

Setting	Yuyao Ding	Pratiksha Kaushik
Epoch definition	Fixed 1,000 optimizer updates; not a full pass through all photos.	Fixed 800 optimizer steps per epoch.
Completed epochs and total updates	100 epochs; 100,000 logged updates.	80 epochs; 64,000 steps (after a 4-configuration × 15-epoch screening).
Precision and random seed	FP32; split seed 41.	bf16 autocast + channels_last; seed 9002.

Independent design differences: Pratiksha used a 2-scale discriminator, a lower identity weight (1 instead of 5), batch 4, EMA generators and DiffAugment, chosen by a 15-epoch screening of four configurations scored with the course metric; the second discriminator scale gave the largest gain. Her final model averages the EMA weights of epochs 60, 65 and 70 instead of using a single checkpoint.

Both reconstruction cycles are explicit: Monet → photo → Monet and photo → Monet → photo. The epoch-70 checkpoint was selected on validation KID. Held-out cycle L1 is 0.066813 (Monet) and 0.061143 (photo) in [0,1] pixel space; fixed input/translation/reconstruction panels permit visual checking.

4.3 Bidirectional Evaluation Results

Protocol in the following table: held-out test (30 images per direction).

The course-protocol results (300 sorted raw images per direction, including training images) are displayed separately below. KID is on its raw scale. Small held-out samples make FID especially noisy; subset standard deviation is not a confidence interval.

Metric	Yuyao A to B	Yuyao B to A	Pratiksha A to B	Pratiksha B to A
Evaluated image count	30	30	300 Monets vs 1,000 held-out photos	1,000 held-out photos vs 300 Monets
FID	176.1675	208.0610	85.95	93.22
Course MiFID	0.418549	0.408350	Not computed on this protocol	Not computed on this protocol
KID and scale	0.018444 raw	0.014324 raw	0.019510 raw (± 0.00220)	0.020630 raw (± 0.00206)
Generative precision	0.9333	0.4333	0.7133	0.4830
Generative recall	0.4333	0.8333	0.4360	0.5900
Density and coverage if used	1.7933 / 0.9333	0.3067 / 0.7667	0.7187 / 0.4510	0.4076 / 0.8367
Cycle-reconstruction L1	0.066813	0.061143	0.0730 (Monet → photo → Monet)	0.1168 (photo → Monet → photo)
LPIPS and image comparison	input → translation 0.293948; input → cycle 0.292868	input → translation 0.388009; input → cycle 0.231483	Input vs translation 0.348; input vs reconstruction 0.287	Input vs translation 0.389; input vs reconstruction 0.210

Metric	Yuyao A to B	Yuyao B to A	Pratiksha A to B	Pratiksha B to A
Content-preservation cosine similarity	0.814094	0.751451	0.7849	0.7343
Translation images per second	70.11	85.80	Not measured separately	505.3 (batch 16, median of 5 runs)

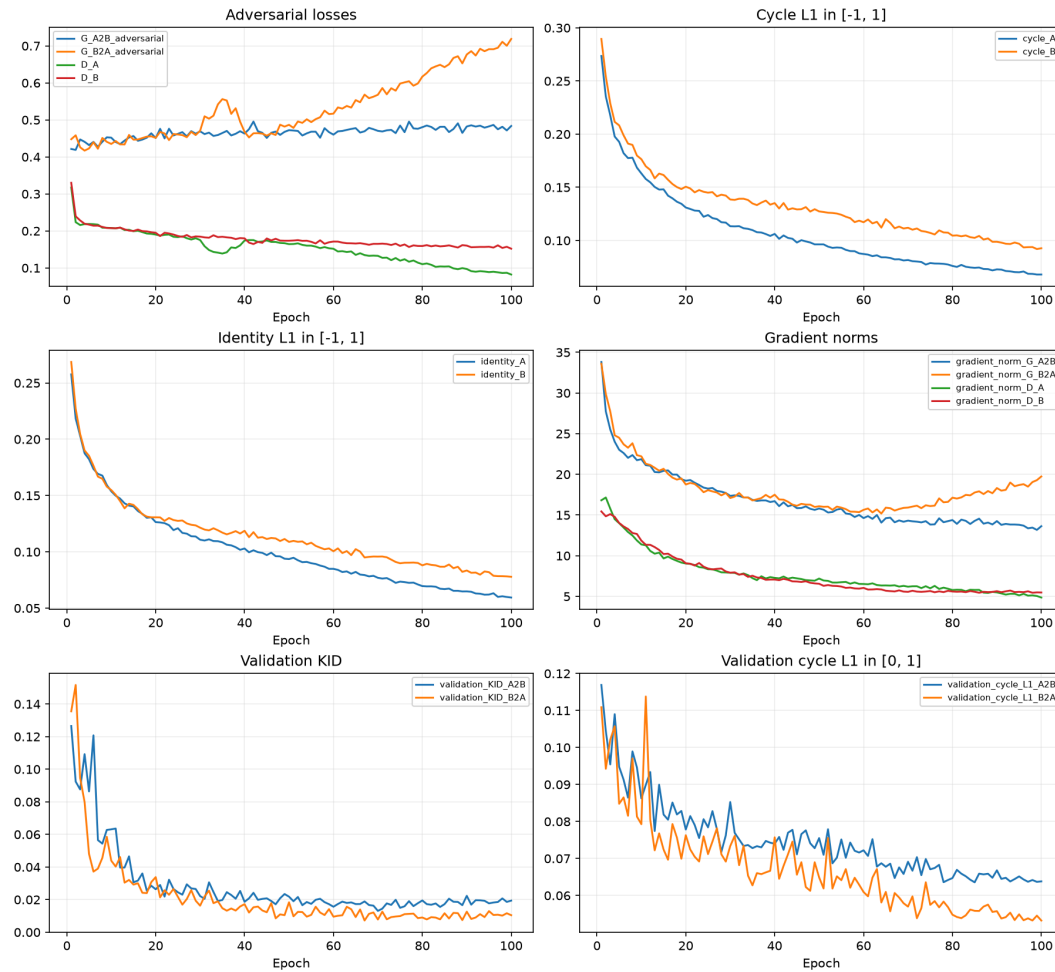
Course evaluation protocol (300 raw images per direction; includes training images)

Metric	Yuyao A to B	Yuyao B to A	Pratiksha A to B	Pratiksha B to A
Evaluated image count	300	300	300	300
FID	103.4908	107.0100	99.576	100.538
Course MiFID	0.424612	0.413559	0.4170	0.4099
KID and scale	0.018389 raw	0.012290 raw	Not computed on this protocol	Not computed on this protocol
Generative precision	0.7400	0.4600	Not computed on this protocol	Not computed on this protocol
Generative recall	0.4733	0.7000	Not computed on this protocol	Not computed on this protocol
Density and coverage if used	0.9400 / 0.8633	0.3500 / 0.6267	Not computed on this protocol	Not computed on this protocol
Cycle-reconstruction L1			Not computed on this protocol	Not computed on this protocol
LPIPS and image comparison			Not computed on this protocol	Not computed on this protocol
Content-preservation cosine similarity	0.831027	0.773844	Not computed on this protocol	Not computed on this protocol
Translation images per second	88.83	84.35	Not computed on this protocol	Not computed on this protocol

4.4 Training Stability and Resource Use

Metric	Yuyao Ding	Pratiksha Kaushik
Generator losses in both directions	Epoch 70: G A → B 0.470378; G B → A 0.567938. Epoch 100: 0.483592 / 0.718688.	Last 25% of steps: adversarial P→M 0.695, M→P 0.519; total generator loss 2.59 (3.97 in the first 25%).
Discriminator losses for both domains	Epoch 70: D Monet 0.132709; D photo 0.165089. Epoch 100: 0.082222 / 0.152003.	Last 25% of steps: D Monet 0.093; D photo 0.152 (both falling after epoch 40).

Metric	Yuyao Ding	Pratiksha Kaushik
Cycle-consistency losses for both domains	Unweighted L1 [-1,1], epoch 70 Monet 0.081600 / photo 0.111967; epoch 100 0.068002 / 0.092694.	Unweighted L1, last 25%: photo cycle 0.068; Monet cycle 0.054 (first 25%: 0.145 / 0.135).
Identity losses for both domains	Unweighted L1 [-1,1], epoch 70 Monet 0.076947 / photo 0.095761; epoch 100 0.059414 / 0.077825.	Unweighted L1, last 25%: Monet 0.077; photo 0.084.
Gradient norms by network	Epoch 70 G A → B/B → A 14.310256/15.918826; D Monet/photo 6.193459/5.558846.	Pre-clip L2 (no clipping used): G median 16.3, p99 33.7, max 146.8; D median 7.3, p99 18.8, max 434.9.
NaN and Inf events	0 in 100,000 logged updates.	0 in 64,000 steps.
Generator and discriminator parameter counts	11,378,179 per generator; 2,764,737 per discriminator; all four 28,285,832.	11,378,179 per generator; 5,529,474 per 2-scale discriminator; all four 33,815,306.
Total training time	18,602.59 s training wall; 18,321.28 s training loops.	11,532.5 s (3.20 h) for the final 80-epoch run; about 6 h including screening.
Training images per second	10.92 image presentations/s across both domains; 5.46 updates/s.	22.2 images/s per domain (44.4 across both domains); 5.55 steps/s.
Peak memory and measurement type	Peak CUDA allocation 2,596.36 MiB; sampled process RSS 3,673.09 MiB.	Peak CUDA allocation 6.33 GiB (training); 2.99 GiB (inference).



Validation mean KID improved from 0.131001 at epoch 1 to 0.010435 at selected epoch 70, then rose to 0.014916 at epoch 100. Cycle/identity components declined; $B \rightarrow A$ adversarial loss rose late while D Monet loss fell, indicating a remaining balance concern. No NaN/Inf events or observed collapse across inspected fixed panels. Late metrics are broadly flat; extending this schedule alone is not supported as a fix.

4.5 Visual Quality and Failure Cases

Examples from both members and both translation directions are shown below, with observations on style transfer, content preservation and artifacts.

Member and direction	Input image	Translation	Cycle reconstruction	Observation
Yuyao A to B	Same-panel input: task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/A2B_0260d15306_input_translation_cycle.png	Same-panel middle translation: task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/A2B_0260d15306_input_translation_cycle.png	Same-panel right cycle: task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/A2B_0260d15306_input_translation_cycle.png	River and tree shapes sharpen somewhat, but painted edges remain diffuse rather than photorealistic.
Yuyao B to A	Same-panel input: task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/B2A_02620a36f7_input_translation_cycle.png	Same-panel middle translation: task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/B2A_02620a36f7_input_translation_cycle.png	Same-panel right cycle: task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/B2A_02620a36f7_input_translation_cycle.png	Sunset shifts to pale yellow/green; water gains repeated marks while the shoreline remains recognizable.
Pratiksha A to B	Top row of task3_gan/Pratiksha_Kaushik/outputs/figures/qual_monet2photo.png (8 Monets)	Middle row of the same figure	Bottom row of the same figure	Colours and contrast become more photographic and the layout is kept, but brush texture often remains; foggy, low-contrast paintings are turned into different scenes (failure_analysis.md, categories 5–6).
Pratiksha B to A	Top row of task3_gan/Pratiksha_Kaushik/outputs/figures/qual_photo2monet.png (8 held-out photos)	Middle row of the same figure	Bottom row of the same figure	Coasts, water and mid-tone landscapes get a pastel palette, soft edges and brush-like texture; night scenes are washed out, smooth skies get blotches and dense texture is lost (failure_analysis.md, categories 1–4).

4.6 Blinded Human Audit

- Fixed sample count: 30 anonymous cases for Yuyao's selected CycleGAN, with 15 Monet → photo and 15 photo → Monet input/translation pairs.
- Both raters scored the same fixed 30 cases. The cases were selected before final evaluation and identified by anonymous case ID.
- Two raters: Liming Jiang and Sherry Tang each completed a separate 30-case rating sheet.
- Blinding protocol: case IDs concealed source filenames and model/checkpoint identity; target style was shown so raters could judge the translation. The instructions requested independent ratings. The saved sheets cannot establish whether the raters discussed their scores.
- Rating scale and criteria: style transfer, content preservation, and freedom from artifacts were each scored from 1 (poor) to 5 (strong); 3 indicated a partial result. Higher scores are better.
- Agreement method: exact agreement and unweighted Cohen's κ were calculated separately for style (S), content (C), and artifact-free quality (A), matching the two raters by case ID. Reported means pool both raters' scores across all 30 cases.

Audited model or set	Style score	Content score	Artifact-free score	Inter-rater agreement
Yuyao	2.67 / 5	2.90 / 5	2.48 / 5	Exact: S 66.7%; C 83.3%; A 66.7% κ : S 0.509; C 0.765; A 0.537
Pratiksha	3.52 / 5	3.93 / 5	3.02 / 5	Exact: S 50.0%; C 60.0%; A 50.0% / κ : S 0.227; C 0.291; A 0.215 (quadratic-weighted κ : S 0.595; C 0.494; A 0.522)

Evidence: task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/audit/ contains instructions, two completed rater CSV files, and blinded image pairs; audit_results.json in the parent evaluation directory records the aggregate results.

Interpretation: content preservation scored highest (2.90/5) and had the strongest agreement (83.3%, $\kappa=0.765$); artifact-free quality scored lowest (2.48/5). Agreement measures rater consistency, not image quality. Findings are limited to these 30 fixed cases.

Pratiksha's audit: 30 fixed samples (20 photo → Monet, 10 Monet → photo) were chosen by seed before final evaluation and shown as anonymous, shuffled input | output pairs, with the direction key kept separately. Two raters scored each sample separately on the same 1–5 criteria. Content preservation scored highest (3.93/5) and artifact-free quality lowest (3.02/5); the raters never differed by more than one point (100% within ± 1), and the low unweighted κ comes mostly from 3-versus-4 splits. Monet → photo was rated higher than photo → Monet on all criteria, but it has only 10 samples. Evidence: task3_gan/Pratiksha_Kaushik/outputs/human_audit/ (rater1.csv, rater2.csv, audit_summary.csv, images/).

4.7 Kaggle Submission and Leaderboard

Competition: DATA 266 Fall 2026 - GAN Image Style Transfer

Team name: PairProgramming_Team_41

Owner and submission ID	Checkpoint	Date	Local FID	Local MiFID	Public score	Rank
Yuyao — submission.csv (CSV row ID=1)	task3_formal_20260927T075834Z_d335ae80; epoch 70	Oct1, 2026	105.250404	0.419085	-52.8347	Individual rank not shown
Pratiksha — final_submission.csv (uploaded as submission.csv)	v3: EMA average of epochs 60, 65 and 70 + Monet → photo colour calibration	Oct 6, 2026	100.056761	0.413453	-50.2351	36 (recorded for the earlier -50.2367 submission)

Selected team submission: Pratiksha's final_submission.csv (uploaded to Kaggle as submission.csv) (CycleGAN v3, EMA average of epochs 60/65/70 with Monet → photo colour calibration), Kaggle Public score -50.2351, the better of the two members' scores.

Yuyao's submission.csv was generated from the selected epoch-70 CycleGAN's own translations using the course 300-image-per-direction evaluation, then averaged across directions. CSV row ID=1 contains local FID 105.250404 and MiFID 0.419085; ID=1 is not a Kaggle submission ID. Her uploaded CSV received Kaggle Public score -52.8347, documented by task3_gan/Yuyao_Ding/outputs/kaggle_public_submissions.png. An individual rank and Private score were not shown. The selected team submission is a separate record.

Local CSV and evaluation record:

task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/.

Pratiksha's final_submission.csv (uploaded to Kaggle as submission.csv) is the v3 notebook's own course-protocol output (FID 100.056761, MiFID 0.413453) and received Kaggle Public -50.2351. An earlier upload of the same predictions scored by a second run (submission_official_v3.csv: FID 100.059940, MiFID 0.413462) received -50.2367. Kaggle shows $-(\text{FID} + \text{MiFID})/2$, so -50.2351 is the better score. Evidence: task3_gan/Pratiksha_Kaushik/outputs/kaggle/kaggle_submission_final.png and kaggle_submission_earlier.png.

4.8 Joint Analysis

Model comparison: On the course protocol, Pratiksha's v3 model scores lower FID in both directions (99.58 / 100.54 vs Yuyao's 103.49 / 107.01) and a better Kaggle Public score (-50.2351 vs -52.8347). Her model has more discriminator capacity (2 scales, 33.8M vs 28.3M parameters in total), a lower identity weight and EMA averaging; Yuyao's has a held-out test split for both domains and stronger content preservation.

Metrics and visual judgments: Pratiksha's audit means (style 3.52, content 3.93, artifacts 3.02) are higher than Yuyao's (2.67, 2.90, 2.48), but the samples, raters and inputs differ, so the audits are not a controlled comparison. Both audits rate content highest and artifacts lowest, matching the metric results: content cosine is high in both directions while both members list colour blotches, smears and texture loss as the main visual failures.

Limitations: Only 300 Monets exist, so Monet-side FID is noisy and uses training paintings as the reference for both members. Each member ran one seed, the evaluation protocols differ (30 vs 1,000 held-out photos), and low cycle L1 does not prove faithful translations, because CycleGAN can hide input information in the output.

Next experiment: Evaluate both final models with one shared held-out protocol and one pair of raters, then test spectral normalization on the discriminators and resize-convolution upsampling, which target the late discriminator dominance and the streak artifacts both members observed.

5 Reproducibility and Artifact Access

5.1 Setup and Smoke Test

Environment instructions: https://github.com/Ashley96yy/DATA266_Pair41_Lab1/blob/main/README.md and the three task-specific Yuyao_Ding/README.md files; pinned task requirements are in each task folder. Pratiksha's environment details are in each task's Pratiksha_Kaushik/README.md and the run manifests under reproducibility/manifests/Pratiksha_Kaushik/; her notebooks run top to bottom in Google Colab on a GPU.

Restore raw datasets and the six Drive artifact folders using https://github.com/Ashley96yy/DATA266_Pair41_Lab1/blob/main/README.md. Place contents into each task's Yuyao_Ding/data_processed/ or checkpoints/ directory without changing run IDs. Pratiksha's checkpoints and large files: restore from the Google Drive links in the repository README ("Pratiksha's work and artifact downloads") into each task's Pratiksha_Kaushik/checkpoints/ directory.

.venv/bin/python task1_llm/Yuyao_Ding/src/smoke_test.py (Mac/Linux; from repository root). Windows equivalent: .\venv\Scripts\python task1_llm/Yuyao_Ding/src/smoke_test.py. Task 2 and 3 have their own smoke_test.py commands in their README files.

Verification item	Recorded outcome
Person who tested reproduction	Yuyao Ding; local smoke test, not an independent reviewer.
Date, operating system, and device	October 5, 2026, 8:24 PM PDT (October 6, 03:24 UTC); macOS 14.6, Apple M3, arm64, MPS; Python 3.12.2 and PyTorch 2.8.0.
Starting conditions	Existing local repository checkout with TinyStories raw text and processed arrays already present; a new Python 3.12 virtual environment. This was not a fresh clone.
Command executed	.venv/bin/python task1_llm/Yuyao_Ding/src/smoke_test.py
Observed result	Model implementation checks passed. The Task 1 smoke run completed one short epoch with three optimizer steps, reloaded its checkpoint, generated greedy and temperature-sampling examples, and printed Smoke test passed. Run ID: smoke_20261006T032413Z_68797f4d.
Output or log evidence	task1_llm/Yuyao_Ding/outputs/mac_smoke_test.log; reproducibility/raw_logs/Yuyao_Ding/smoke_20261006T032413Z_68797f4d.jsonl; matching manifest and generation record. The evidence was pushed in GitHub commit 632a0d8.
Known limitations	This verifies the Task 1 smoke path on the existing Mac checkout. It is not a clean-clone reproduction, a full 10-epoch training run, or a formal metric reevaluation.

5.2 Evidence Index

Task and member	Code and configuration	Metrics and analysis	Original logs and manifests	Checkpoints and processed data
Task 1 Yuyao	task1_llm/Yuyao_Ding/README.md; configs/; src/	task1_llm/Yuyao_Ding/results.md; metrics_report.csv; failure_analysis.md	reproducibility/raw_logs/Yuyao_Ding/; reproducibility/manifests/Yuyao_Ding/	task1_llm/Yuyao_Ding/README.md (Google Drive restoration links)
Task 1 Pratiksha	task1_llm/Pratiksha_Kaushik/README.md; configs/config.json; src/Task1_gpt.ipynb	task1_llm/Pratiksha_Kaushik/results.md; metrics_report.csv; failure_analysis.md	task1_llm/Pratiksha_Kaushik/RUN_LOG.txt; reproducibility/raw_logs/Pratiksha_Kaushik/; reproducibility/manifests/Pratiksha_Kaushik/	task1_llm/Pratiksha_Kaushik/checkpoints/README.md and data_processed/README.md (Google Drive links)
Task 2 Yuyao	task2_sentiment/Yuyao_Ding/README.md; configs/; src/	task2_sentiment/Yuyao_Ding/results.md; metrics_report.csv; failure_analysis.md	reproducibility/raw_logs/Yuyao_Ding/; reproducibility/manifests/Yuyao_Ding/	task2_sentiment/Yuyao_Ding/README.md (Google Drive restoration links)
Task 2 Pratiksha	task2_sentiment/Pratiksha_Kaushik/README.md; configs/; src/task2_yelp_sentiment_all_models.ipynb	task2_sentiment/Pratiksha_Kaushik/results.md; metrics.md; metrics_report.csv; failure_analysis.md; manual_error_review.md	task2_sentiment/Pratiksha_Kaushik/RUN_LOG.txt; reproducibility/raw_logs/Pratiksha_Kaushik/; reproducibility/manifests/Pratiksha_Kaushik/	task2_sentiment/Pratiksha_Kaushik/checkpoints/README.md (Google Drive link)
Task 3 Yuyao	task3_gan/Yuyao_Ding/README.md; configs/; src/	task3_gan/Yuyao_Ding/results.md; metrics_report.csv; failure_analysis.md	reproducibility/raw_logs/Yuyao_Ding/; reproducibility/manifests/Yuyao_Ding/	task3_gan/Yuyao_Ding/README.md (Google Drive restoration links)
Task 3 Pratiksha	task3_gan/Pratiksha_Kaushik/README.md; configs/; src/; evaluate_local.py	task3_gan/Pratiksha_Kaushik/results.md; full_metrics_report.csv; failure_analysis.md	task3_gan/Pratiksha_Kaushik/RUN_LOG.txt; reproducibility/raw_logs/Pratiksha_Kaushik/; reproducibility/manifests/Pratiksha_Kaushik/	task3_gan/Pratiksha_Kaushik/checkpoints/README.md (Google Drive links)

6 Team Conclusions and References

6.1 Team Conclusions

Paragraph 1 Main findings: Both members built every required component from scratch and trained stable models with no non-finite steps. Model scale and data size drove most of the differences: Pratiksha's larger Task 1 model reached 0.684 bits/character against 1.273, and Yuyao's larger Task 2 training set gave 0.9475 against 0.9054 accuracy. In Task 2 a CNN beat the mean-pooling baseline significantly for both members, and in Task 3 Pratiksha's 2-scale discriminator setup reached the better Kaggle Public score (-50.2351 vs -52.8347).

Paragraph 2 Trade-offs and limitations: Better scores came with higher cost: Pratiksha's Task 1 model needs about 15× the memory of Yuyao's and generates slower, and Yuyao's BiLSTM trains about 5× longer than her baseline. Every comparison rests on one seed per model, and the members' splits, test sets and evaluation protocols differ, so between-member differences are descriptive rather than controlled.

Paragraph 3 Next steps: Rerun the key comparisons on shared splits with several seeds; in Task 1 add a KV-cache and a size sweep, in Task 2 fix the preprocessing issues both members found (removed contrast and comparison words, length cutoff), and in Task 3 evaluate both CycleGANs with one held-out protocol and one pair of raters.

6.2 References

References used in this report:

Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems 30. <https://arxiv.org/abs/1706.03762>

Eldan, R., & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? <https://arxiv.org/abs/2305.07759>

Zhu, J.-Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks. ICCV. <https://arxiv.org/abs/1703.10593>

Original TinyStories, Yelp Polarity, and Monet/photo dataset source and access instructions: repository README.md, "Raw datasets."

Course Part3_Evaluation_Script.ipynb; Yuyao Task 3 README.md documents the compatibility change and evaluation definitions.

Appendix A Complete Task 2 Error Reviews

Yuyao's completed 60 model-case entries and Pratiksha's 20 CNN entries are listed below, with exact snippets, labels, probabilities, observations, and proposed checks. The linked `failure_analysis.md` and `manual_error_review.md` files retain the full text and evidence.

Review Coverage

Member and model	Confident false positives	Confident false negatives	Near-threshold errors	Slice-specific failures
Yuyao Baseline	test_4742, test_17640, test_20681, test_3163, test_23566	test_1131, test_24208, test_22707, test_16998, test_29123	test_24137, test_12064, test_868, test_9057, test_13888	test_9438, test_6520, test_22807, test_31102, test_26639
Yuyao TextCNN	test_37771, test_29330, test_11176, test_4655, test_18122	test_16801, test_30793, test_16532, test_29494, test_3223	test_33465, test_3618, test_19442, test_28747, test_13462	test_26172, test_20061, test_14906, test_22807, test_30958
Yuyao BiLSTM	test_10827, test_23764, test_2603, test_34683, test_18122	test_22807, test_10288, test_30958, test_30793, test_9871	test_16678, test_28480, test_34705, test_32562, test_31449	test_22206, test_20061, test_37988, test_30086, test_37771
Pratiksha CNN	P1, P2, P3, P4, P5	P6, P7, P8, P9, P10	P11, P12, P13, P14, P15	P16, P17, P18, P19, P20

Individual Case Record — representative example

Member and model: Yuyao Ding, baseline (Y-B)

Case ID and category: test_17640, confident false positive

Actual review snippet: "my least favorite"

True label: negative; predicted: positive; $P(\text{positive})=1$

Error type: negative comparison lost in preprocessing

Analysis: "least" disappears from processed text; processed input becomes "buffets vegas favorite."

Testable fix: preserve comparison operators; compare validation errors before any new test evaluation.

Evidence: task2_sentiment/Yuyao_Ding/failure_analysis.md, baseline case 2.

Baseline — test_4742 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=1$. Exact excerpt: "less than accomodating or helpful"

Observed: Preprocessing: lost comparison. — The processed input is only 'accomodating helpful'. Removing 'less than' reverses the meaning available to the classifier.

Proposed validation check: Keep comparison words such as less and than; retrain with the same split and compare validation errors on comparative phrases.

Baseline — test_17640 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=1$. Exact excerpt: "my least favorite"

Observed: Preprocessing: lost comparison. — The processed input is 'buffets vegas favorite'. The word 'least', which makes this a negative comparison, has been removed.

Proposed validation check: Preserve least and other comparison operators, then compare validation accuracy with the original stopword rule.

Baseline — test_20681 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99999714$. Exact excerpt: "Large variety"

Observed: Ambiguous text-label relation. — The complete review contains only these two apparently positive words, but the supplied label is negative. The text alone does not explain the rating.

Proposed validation check: Audit similarly short validation reviews against their source ratings where available; retain the official test label and report short-text uncertainty.

Baseline — test_3163 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.999997$. Exact excerpt: “That is all I love though...”

Observed: Restricted praise. — The final sentence limits the praise to beer. The processed input is 'love rubios beer n nthat love', so the restriction is largely lost and praise is repeated.

Proposed validation check: Compare lighter stopword removal and sentence-aware pooling on validation reviews containing limited praise.

Baseline — test_23566 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99999356$. Exact excerpt: “fast, a little toooo fast”

Observed: Implicit criticism. — The repeated speed word looks positive in isolation, but 'a little toooo fast' can criticize rushed service. The supplied negative label depends on that qualification.

Proposed validation check: Evaluate character or subword features and repeated-letter normalization on validation examples with informal intensifiers.

Baseline — test_1131 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=1.1857003e-08$. Exact excerpt: “wont do ya dirty”

Observed: Informal negation and idiom. — The apostrophe-free 'wont' is not expanded by the contraction rule. The input 'margaritas wont ya dirty' also requires the positive idiom 'do no harm'.

Proposed validation check: Normalize common apostrophe-free contractions and test phrase or character features on informal validation text.

Baseline — test_24208 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=2.1770502e-06$. Exact excerpt: “is the restaurant closed?”

Observed: Question rather than sentiment. — This asks about opening status rather than clearly expressing dissatisfaction. Processing leaves 'not restaurant closed', while the supplied label is positive.

Proposed validation check: Inspect question and business-status slices on validation data and distinguish factual closure mentions from sentiment.

Baseline — test_22707 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=2.4895903e-06$. Exact excerpt: “but variety was just ok.... Meh..”

Observed: Mixed sentiment and rating ambiguity. — The review praises the view but ends with weak food praise. A negative prediction is understandable from the wording even though the supplied label is positive.

Proposed validation check: Audit mixed validation reviews and compare sentence-level aggregation without changing official test labels.

Baseline — test_16998 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=9.54978e-06$. Exact excerpt: “are you gonna reopen?”

Observed: Implicit affection in a complaint. — Repeated requests to reopen can imply that the reviewer misses the business. The model instead receives words such as 'wtf' and 'reopen' with little explicit praise.

Proposed validation check: Evaluate a validation slice of reopening questions and informal expressions; compare phrase-aware features.

Baseline — test_29123 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=1.7520259e-05$. Exact excerpt: “made me give it four stars”

Observed: Preprocessing: lost rating cue. — The explicit four-star explanation supports the positive label. Processing reduces the whole review to 'ok but bread stars', dropping the number word 'four'.

Proposed validation check: Preserve number words and explicit rating phrases, then test the change on validation data.

Baseline — test_24137 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.4999231$. Exact excerpt: “I recommend it”

Observed: Mixed sentiment and comparison. — Complaints about Vegas prices and portions surround a recommendation for this restaurant. Mean pooling cannot directly assign those statements to different comparison targets.

Proposed validation check: Compare sentence or attention pooling and evaluate mixed-review validation cases using the same training split.

Baseline — test_12064 (Near threshold)

Label negative; predicted positive; $P(\text{positive})=0.5004356$. Exact excerpt: “It just wasn't worth the price”

Observed: Positive setting but negative value. — Praise for the atmosphere competes with tiny portions, high cost and a negative final verdict. The prediction is only just above the fixed threshold.

Proposed validation check: Test sentence-aware pooling that retains the final verdict; assess any threshold change on validation data only.

Baseline — test_868 (Near threshold)

Label negative; predicted positive; $P(\text{positive})=0.5006376$. Exact excerpt: “We spent most of our night waiting in line”

Observed: Aspect-specific disappointment. — The drinks and company are praised, but the event organization is criticized. Averaging positive and negative words nearly cancels the overall complaint.

Proposed validation check: Compare sentence-level aggregation on validation reviews where the product is good but the experience is poor.

Baseline — test_9057 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.49926805$. Exact excerpt: “a near- Nirvana experience with every bite”

Observed: Positive outcome after inconvenience. — The long wait and brusque service are explicitly outweighed by the food. All 92 processed tokens fit, so this is not a truncation error.

Proposed validation check: Compare context-aware pooling on validation narratives whose conclusion reverses earlier complaints.

Baseline — test_13888 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.49921444$. Exact excerpt: “Overall, this is a good music venue”

Observed: Mixed aspects with positive conclusion. — The review recommends the venue despite seating complaints. All 130 processed tokens fit; missing tail text cannot explain this case.

Proposed validation check: Test sentence-level weighting while keeping the same split and inspect venue reviews with minor complaints.

Baseline — test_9438 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=3.178318e-05$. Exact excerpt: “never have to go to ghetto Fiesta Mall”

Observed: Wrong comparison target. — The negative language describes alternative malls, while the reviewer wants to keep using the reviewed mall. A bag of words loses this target distinction.

Proposed validation check: Evaluate comparison-target examples on validation data and compare a sequence-aware baseline.

Baseline — test_6520 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=5.0243525e-05$. Exact excerpt: “The beer is Delicious and so is the food”

Observed: Mixed aspects and rating ambiguity. — Positive food and atmosphere statements are followed by strong service complaints. The positive supplied label weights those aspects differently from the negative prediction.

Proposed validation check: Compare aspect or sentence aggregation and audit mixed validation labels instead of relabeling the test case.

Baseline — test_22807 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=8.298872e-05$. Exact excerpt: “They really did change the service up”

Observed: Updated review versus historical complaint. — A short positive edit precedes a much longer older complaint. All 52 processed tokens fit. Each model's selected error shows difficulty separating the update from the history.

Proposed validation check: Preserve update and sentence boundaries and compare update-aware pooling on validation examples; do not assume a longer sequence alone will fix it.

Baseline — test_31102 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=9.7369295e-05$. Exact excerpt: “has not closed”

Observed: Negation scope. — The review corrects a closure rumor and says the restaurant is open. 'not' survives preprocessing, so retaining the token alone did not make the mean-pooling model learn its scope.

Proposed validation check: Test negation-aware bigrams or a sequence model on a fixed validation slice of business-status corrections.

Baseline — test_26639 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.00012118082$. Exact excerpt: “Need a refund on our \$20 voucher”

Observed: Ambiguous text-label relation. — The text describes closure and inconvenience, despite its positive supplied label. The reason for that label is not recoverable from this excerpt alone.

Proposed validation check: Audit similar validation labels against original ratings where available; retain this official test label and avoid claiming certain annotation error.

TextCNN — test_37771 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99987245$. Exact excerpt: “DO NOT EAT AT THIS LOCATION”

Observed: Chain praise versus location complaint. — Praise for the chain is followed by an explicit warning about this branch. All 31 processed tokens fit, so both selected models miss a target and contrast distinction rather than omitted text.

Proposed validation check: Test sentence-level target and contrast aggregation on validation reviews that compare a chain with one branch.

TextCNN — test_29330 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99978596$. Exact excerpt: “Great place to come and relax worth a try!”

Observed: Ambiguous text-label relation. — The complete text is positive, but the supplied label is negative. There is no clear negative statement to use as an explanation.

Proposed validation check: Audit comparable validation examples against source ratings; leave official test labels unchanged and report the ambiguity.

TextCNN — test_11176 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99962795$. Exact excerpt: “Ruined by music”

Observed: Desired state versus actual experience. — The poem says the place should be calm and tranquil, not that it was. Positive descriptive words refer to a desired state and may compete with the actual complaint.

Proposed validation check: Keep modal words and sentence boundaries; test this preprocessing change on hypothetical or desired-state validation reviews.

TextCNN — test_4655 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.9994824$. Exact excerpt: “totally the coolest place, like, ever”

Observed: Mixed sentiment and possible sarcasm. — The review lists good and bad points and mocks the crowd's excitement. Literal positive phrases could mislead max pooling; sarcasm is a plausible reading, not a measured attribution.

Proposed validation check: Compare sentence-aware aggregation on a reviewed validation subset containing irony and mixed pros and cons.

TextCNN — test_18122 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99938834$. Exact excerpt: “sad to see them go”

Observed: Closure sadness versus quality praise. — The reviewer mourns a closure while repeatedly praising the food and service. The negative supplied label may reflect sadness rather than poor business quality.

Proposed validation check: Audit closure and farewell validation reviews and separate emotional tone from evaluation of the business.

TextCNN — test_16801 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.0004381646$. Exact excerpt: “will always return”

Observed: Mixed review with return intention. — The reviewer intends to return and likes the sandwich, but several sides and billing are criticized. The selected CNN prediction is dominated by an overall negative decision despite these positive commitments.

Proposed validation check: Compare sentence or attention pooling and evaluate return-intention validation examples without changing the test threshold.

TextCNN — test_30793 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.0005658463$. Exact excerpt: “Now its much better”

Observed: Past versus current experience. — The old owners receive strong criticism, whereas the new owners are praised at both ends of the review. All 46 processed tokens fit; this is a temporal contrast problem.

Proposed validation check: Keep temporal markers and compare sentence-aware context aggregation on before-and-after validation reviews.

TextCNN — test_16532 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00059241685$. Exact excerpt: “Great gym, but”

Observed: Mixed sentiment and rating ambiguity. — A short positive opening is followed by a detailed cancellation and billing complaint. The supplied positive label is not an obvious summary of the full text.

Proposed validation check: Audit cancellation-related validation reviews and compare aspect-level summaries, keeping official labels fixed.

TextCNN — test_29494 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.0008924034$. Exact excerpt: “updating my review to 4 stars”

Observed: Update and service recovery. — The long billing complaint is followed by a successful resolution and an explicit positive rating update. All 165 processed tokens fit, so this cannot be attributed to the 192-token limit.

Proposed validation check: Preserve update and rating cues and compare sentence-level aggregation on validation cases with resolved complaints.

TextCNN — test_3223 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.0009119775$. Exact excerpt: “will probably go back again”

Observed: Mixed visits and return intention. — A good first visit and possible return coexist with a disappointing second visit. The wording gives genuine mixed evidence despite the positive label.

Proposed validation check: Evaluate multiple-visit validation reviews and compare sentence aggregation with max pooling.

TextCNN — test_33465 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.49974895$. Exact excerpt: “Not bad all. Love the beers”

Observed: Negation scope. — The short review expresses praise through 'not bad'. The CNN output lies just below 0.5 even though the negation token is retained.

Proposed validation check: Compare negation-aware phrase features on validation text; select any calibration or threshold adjustment using validation labels only.

TextCNN — test_3618 (Near threshold)

Label negative; predicted positive; $P(\text{positive})=0.50026375$. Exact excerpt: “This company will rob local shoppers”

Observed: Promotion language inside a complaint. — The review quotes an attractive promotion to criticize its restrictions. Positive sales words describe the disputed offer, not the reviewer's approval.

Proposed validation check: Preserve quotation and sentence structure and compare context-aware pooling on promotion-complaint validation cases.

TextCNN — test_19442 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.49969393$. Exact excerpt: “I'd try a few more tapases”

Observed: Mixed aspects with truncated ending. — The review mixes praise and criticism. Its 252 processed tokens exceed 192, excluding the conclusion and some service discussion; this is a plausible source of lost context.

Proposed validation check: Compare head-plus-tail input or a longer limit on validation data and report changes specifically on truncated reviews.

TextCNN — test_28747 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.49960852$. Exact excerpt: “worst kept secret in Vegas”

Observed: Idiomatic wording and mixed aspects. — The opening uses a negative-looking idiom to describe a popular restaurant, then praises the pizza while noting queues and price. All 81 processed tokens fit.

Proposed validation check: Compare contextual phrase handling on validation examples with idioms, keeping the existing test set untouched.

TextCNN — test_13462 (Near threshold)

Label negative; predicted positive; $P(\text{positive})=0.5005151$. Exact excerpt: “If u want a guy threatening you”

Observed: Safety complaint among amenities. — Cheap drinks and friendly staff are mentioned, but threats are central to the negative conclusion. The prediction falls only slightly above 0.5.

Proposed validation check: Evaluate aspect and sentence weighting on validation reviews where a serious complaint outweighs minor amenities.

TextCNN — test_26172 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0014236559$. Exact excerpt: “would eat here more often”

Observed: Wrong comparison target. — The harsh criticism concerns grocery-store instant ramen, while the restaurant is recommended. All 80 processed tokens fit, so target confusion is more plausible than truncation.

Proposed validation check: Test comparison-target validation cases with sentence or attention pooling.

TextCNN — test_20061 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0014349369$. Exact excerpt: “Can't beat that!”

Observed: Domain vocabulary and truncated resolution. — Garbage and waste are literal descriptions of the service. Only 192 of 290 processed tokens are retained, omitting the final resolution and positive conclusion.

Proposed validation check: Compare head-plus-tail input on validation data and inspect non-restaurant domains separately before attributing all errors to length.

TextCNN — test_14906 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0015368646$. Exact excerpt: “SouthPointe matched the lower price”

Observed: Past complaint versus successful resolution. — Much of the complaint concerns a prior booking site and last year's experience. The current hotel resolves the price problem; all 69 processed tokens fit.

Proposed validation check: Compare sentence-aware handling of entity and time changes on validation booking reviews.

TextCNN — test_22807 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0016308617$. Exact excerpt: “They really did change the service up”

Observed: Updated review versus historical complaint. — A short positive edit precedes a much longer older complaint. All 52 processed tokens fit. Each model's selected error shows difficulty separating the update from the history.

Proposed validation check: Preserve update and sentence boundaries and compare update-aware pooling on validation examples; do not assume a longer sequence alone will fix it.

TextCNN — test_30958 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0017511233$. Exact excerpt: “why do I give this store 5 stars?”

Observed: Company complaint versus local service recovery. — The local manager resolves a long complaint about the wider company. There are 193 processed tokens, but truncation removes only the final 'thank'; the positive resolution remains visible.

Proposed validation check: Test entity and resolution-aware sentence pooling on validation data. Do not treat a longer limit as a sufficient fix for this case.

BiLSTM — test_10827 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99916995$. Exact excerpt: “The music sucked”

Observed: Mixed aspects and emotional emphasis. — Praise for a dancer, lockers and one song competes with strong criticism of the club's music. All 88 processed tokens fit; the positive decision misses the review's weighting of those aspects.

Proposed validation check: Compare sentence or aspect aggregation on mixed club-review validation examples.

BiLSTM — test_23764 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99894756$. Exact excerpt: “it's not bad, but it's not amazing either”

Observed: Pros and cons with weak food verdict. — The review lists many amenities but criticizes food quality. All 189 processed tokens fit, so the long-review slice does not imply truncation in this example.

Proposed validation check: Preserve pros-and-cons boundaries and compare sentence-level aggregation on validation reviews with mixed food and service judgments.

BiLSTM — test_2603 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99894446$. Exact excerpt: “the prices are crazy high”

Observed: Positive exception within a negative comparison. — Shoes are praised as an exception, but prices and overall selection are criticized and competitors are preferred. The positive prediction misses that overall comparison.

Proposed validation check: Evaluate exception and competitor comparisons on validation data with sentence-aware pooling.

BiLSTM — test_34683 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99884593$. Exact excerpt: “They only have one issue, bland, plain food”

Observed: Many amenities versus a decisive food complaint. — Many positive amenities precede the central food complaint and a preference for other restaurants. All 59 processed tokens fit.

Proposed validation check: Compare aspect-aware aggregation on validation cases where one important weakness determines the overall rating.

BiLSTM — test_18122 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.99839395$. Exact excerpt: “sad to see them go”

Observed: Closure sadness versus quality praise. — The reviewer mourns a closure while repeatedly praising the food and service. The negative supplied label may reflect sadness rather than poor business quality.

Proposed validation check: Audit closure and farewell validation reviews and separate emotional tone from evaluation of the business.

BiLSTM — test_22807 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00015927476$. Exact excerpt: “They really did change the service up”

Observed: Updated review versus historical complaint. — A short positive edit precedes a much longer older complaint. All 52 processed tokens fit. Each model's selected error shows difficulty separating the update from the history.

Proposed validation check: Preserve update and sentence boundaries and compare update-aware pooling on validation examples; do not assume a longer sequence alone will fix it.

BiLSTM — test_10288 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.000741355$. Exact excerpt: “trying to make it right and apologizing”

Observed: Service recovery despite food complaints. — The reviewer rewards the staff's response despite disliking the food and water service. All 68 processed tokens fit; the model must weigh resolution against complaints.

Proposed validation check: Evaluate service-recovery validation reviews and compare sentence-level attention.

BiLSTM — test_30958 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00078077434$. Exact excerpt: “why do I give this store 5 stars?”

Observed: Company complaint versus local service recovery. — The local manager resolves a long complaint about the wider company. There are 193 processed tokens, but truncation removes only the final 'thank'; the positive resolution remains visible.

Proposed validation check: Test entity and resolution-aware sentence pooling on validation data. Do not treat a longer limit as a sufficient fix for this case.

BiLSTM — test_30793 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00078439724$. Exact excerpt: “Now its much better”

Observed: Past versus current experience. — The old owners receive strong criticism, whereas the new owners are praised at both ends of the review. All 46 processed tokens fit; this is a temporal contrast problem.

Proposed validation check: Keep temporal markers and compare sentence-aware context aggregation on before-and-after validation reviews.

BiLSTM — test_9871 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.0011509965$. Exact excerpt: “TI is always a good value”

Observed: Narrative trouble with positive outcome. — The hotel resolves the reviewer's booking mistake and receives a positive overall judgment despite several inconveniences. All 136 processed tokens fit.

Proposed validation check: Compare sentence-level summaries on validation narratives with a positive outcome after an initially negative incident.

BiLSTM — test_16678 (Near threshold)

Label negative; predicted positive; $P(\text{positive})=0.500563$. Exact excerpt: “I couldn't wait to leave”

Observed: Minor positive aspect versus negative verdict. — The children's enjoyment of a fountain is a small positive within a tourist-trap complaint. The prediction is just above the threshold despite the final negative verdict.

Proposed validation check: Test sentence-aware pooling on validation cases where the final verdict differs from one praised aspect.

BiLSTM — test_28480 (Near threshold)

Label negative; predicted positive; $P(\text{positive})=0.5007261$. Exact excerpt: “cheaper, tastier, larger burger”

Observed: Wrong comparison target. — These positive adjectives describe the recommended competitor, while the reviewed business is called overpriced. The model crosses the threshold in the wrong direction.

Proposed validation check: Compare target-aware sentence representations on competitor-comparison validation reviews.

BiLSTM — test_34705 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.49916467$. Exact excerpt: “They're the tits”

Observed: Positive slang. — The final slang expression is praise in this context. The review is short and its positive meaning depends on recognizing that idiom.

Proposed validation check: Evaluate slang and idiom validation examples; compare character or phrase features learned only from training data.

BiLSTM — test_32562 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.49902654$. Exact excerpt: “Sandwiches are good”

Observed: Mixed service complaint and compensation. — Good sandwiches and free replacements coexist with repeated order mistakes and limited lunch time. The model is uncertain and lands just below 0.5.

Proposed validation check: Audit mixed service-recovery validation examples and evaluate sentence aggregation; do not tune the threshold on this test case.

BiLSTM — test_31449 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.49883115$. Exact excerpt: “deciding between 4 and 5 stars”
Observed: Mixed rated aspects and truncation. — The review gives an overall positive rating but mixed item ratings. Only 192 of 285 processed tokens are retained; omitted text contains both further criticism and beer praise, so truncation is not a proven sole cause.

Proposed validation check: Compare head-plus-tail input and retention of number-word rating cues on validation data, varying one choice at a time.

BiLSTM — test_22206 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0012314891$. Exact excerpt: “Serendipity, I will be back!”

Observed: Mixed dishes and explanatory negation. — An average quesadilla, a praised drink and a clarification of what the drink is not appear together. All 115 processed tokens fit, including the intention to return.

Proposed validation check: Compare sentence-aware aggregation on validation examples with product explanations and explicit return intent.

BiLSTM — test_20061 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0012814221$. Exact excerpt: “Can't beat that!”

Observed: Domain vocabulary and truncated resolution. — Garbage and waste are literal descriptions of the service. Only 192 of 290 processed tokens are retained, omitting the final resolution and positive conclusion.

Proposed validation check: Compare head-plus-tail input on validation data and inspect non-restaurant domains separately before attributing all errors to length.

BiLSTM — test_37988 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0013292707$. Exact excerpt: “However!!! I experienced something much better”

Observed: Expectation versus actual experience. — The opening negative terms describe expectations based on another branch; the actual visit is praised. All 38 processed tokens fit.

Proposed validation check: Preserve contrast markers and test expectation-versus-experience validation reviews with sentence-aware features.

BiLSTM — test_30086 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0015603743$. Exact excerpt: “a good chef with a bad location”

Observed: Business closure versus product quality. — The review reports a closure and criticizes business planning while praising the chef. The supplied positive label and largely negative situation describe different aspects.

Proposed validation check: Audit closure-related validation examples and distinguish factual business status from food-quality sentiment.

BiLSTM — test_37771 (Long-review / negation slice)

Label negative; predicted positive; $P(\text{positive})=0.9981001$. Exact excerpt: “DO NOT EAT AT THIS LOCATION”

Observed: Chain praise versus location complaint. — Praise for the chain is followed by an explicit warning about this branch. All 31 processed tokens fit, so both selected models miss a target and contrast distinction rather than omitted text.

Proposed validation check: Test sentence-level target and contrast aggregation on validation reviews that compare a chain with one branch.

CNN (P-E1) — P1 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.999901$. Exact excerpt: “of all the buffets I had in Vegas this was my least favorite!”

Observed: CMP (comparative / superlative negative). — after preprocessing, the model sees only all buffets had vegas least favorite. "favorite" is one of the strongest positive words in Yelp reviews. "least" flips it, but "least favorite" is rare compared with "favorite" on its own, so the CNN's 3-gram filters fire on the positive word. The review is only 6 tokens long, so nothing else can outweigh it.

Proposed validation check: build a check set of about 50 test reviews containing "least favorite", "least impressive" or "worst of all". Compare the error rate on this set with the CNN as-is and with a CNN that gets an extra bigram feature (least_favorite, least_impressive). Success means the error rate on the check set drops with no loss in overall macro-F1.

CNN (P-E1) — P2 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.999758$. Exact excerpt: "Dr.Adams is great! Rest of office parley run. Long wait time, violations of patient ..."

Observed: MIX (mixed sentiment). — "is great!" is a very strong positive phrase, and the CNN uses max pooling, so one strong positive feature can dominate. The complaints ("long wait time", "violations of patient privacy", "loud staff") are spread out and use words that aren't strongly negative on their own. The reviewer's overall verdict is negative (praise for one person, complaints about the rest), but the model has no idea which part matters more.

Proposed validation check: score each sentence separately with the same CNN and average the sentence probabilities instead of scoring the whole review at once. Measure the false-positive count on test reviews that have both a positive and a negative sentence. Success means fewer false positives on this subset.

CNN (P-E1) — P3 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.999704$. Exact excerpt: "Good location and friendly staff. But need add more classes for what they charge a month"

Observed: CON (lost contrast word). — the first sentence is clearly positive ("good location", "friendly staff"). The complaint comes after "But", and "but" is in my stopwords list, so it is deleted before the model sees the review. Without it, the complaint ("need add more classes for what they charge") is a neutral-sounding phrase, so the positive opening wins.

Proposed validation check: remove "but" (and keep "however") from the stopwords list and retrain the CNN with everything else the same. Compare the false-positive count on test reviews that contain "but". Success means fewer false positives on that subset, and no drop in overall macro-F1.

CNN (P-E1) — P4 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.999635$. Exact excerpt: "By far, the least impressive Cirque du Soleil show I have seen. The performers are all ..."

Observed: MIX (mixed sentiment), with CMP and CON. — this review packs in many strong positive words: "amazing", "excellent seats", "very comfortable", "really cool". The negative verdict comes from "least impressive" (a superlative negative, as in Error 1), "not showcased or directed well" and "drag on", and it is split by two "but"s that the model never sees. Counting phrases, the positives win.

Proposed validation check: the same "keep but" experiment as Error 3. In addition, compare the CNN and the BiGRU only on test reviews containing "but": the BiGRU reads the review in order, so it should handle the turn better. If the BiGRU has a lower error rate on this subset, it supports combining the two models.

CNN (P-E1) — P5 (Confident false positive)

Label negative; predicted positive; $P(\text{positive})=0.999434$. Exact excerpt: "Attended the @SpaFitFinder launch party, with @spacephx. I can't say much about the ..."

Observed: LBL (possible label noise), with MIX. — most of the text is positive ("really delicious", "very nice gentleman", "very gracious", "everything he brought was delicious", "I enjoyed myself"). The only negatives are "felt cramped" and "not likely I will return". The label is negative (1-2 stars), probably because the reviewer is rating the venue, not the event. A human reading the text would also guess positive, so the model's mistake is understandable.

Proposed validation check: count how many test errors look like this. Take a random sample of 50 confident false positives and mark each one as "text supports its label" or "text doesn't support its label". If more than about 20% don't, part of the error rate is label noise that no model fix can remove, and that should be reported as a limit on accuracy.

CNN (P-E1) — P6 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00098174$. Exact excerpt: "Great gym, but if you ever try to cancel your agreement count on paying 2-3 months longer ..."

Observed: MIX (mixed sentiment), with BIAS and LBL. — the reviewer says "Great gym" once and then spends the whole review on a billing complaint ("cancel" three times, "still getting charged months later"). These are words the model has learned from many negative reviews. The label is positive, presumably because the reviewer still rated the gym itself well. From the text alone, negative is the more natural reading.

Proposed validation check: check the topic-word bias directly. Take test reviews containing "cancel" or "charged" and compare their false-negative rate with the overall false-negative rate. If it is much higher, try down-weighting these words (for example, train with them masked 50% of the time) and check whether the gap shrinks.

CNN (P-E1) — P7 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00112275$. Exact excerpt: "I had dinner here last weekend, and decided to order the gnocchi since I had it at ..."

Observed: MIX (mixed sentiment), with an explicit rating the model can't use. — the story is mostly negative ("overcooked", "dry", "wasn't much better", "didn't like my food", "annoying"). The reviewer states the verdict directly ("I still give them 3 1/2 stars"), but my preprocessing removes punctuation and the model has no idea that "3 1/2 stars" is a rating. "pleasant" and "liked" at the end are too weak to balance the rest.

Proposed validation check: add a simple rule-based feature that finds "N stars" / "N 1/2 stars" in the text and passes the number to the classifier. Measure accuracy on test reviews that mention a star rating, with and without the feature. Success means a clear accuracy gain on that subset.

CNN (P-E1) — P8 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00209297$. Exact excerpt: "For all the people who sit down and don't get any \"\"service,\"\" there are signs that ..."

Observed: SAR (sarcasm / indirect tone). — the review is written as advice to other customers, not as praise. It contains "don't get any service" (a phrase that is almost always negative in Yelp), and the positive meaning is only implied: the recommended list of beer, wings and cheese curds. There are no normal positive words.

Proposed validation check: this is hard for any model that reads phrases. As a measurable step, count how many test reviews quote a complaint from other people ("people who ... don't get ...") and check the error rate on them. If it is high, it should be reported as a known weakness rather than fixed.

CNN (P-E1) — P9 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00226932$. Exact excerpt: "So instead of going to Gibson Automotive to get my wifes battery swapped out we had ..."

Observed: ENT (wrong target). — the review is about Gibson Automotive, and it is positive about Gibson. But most of the text describes a bad experience at a different shop (Pep Boys): "scam artists", "wanted over \$460", "do not trust". The model has no idea which business is being reviewed, so it scores the overall tone, which is negative.

Proposed validation check: raise MAX_LEN from 180 to 400 to check that the end isn't being cut. If that doesn't change the prediction, test a sentence-level model that gives more weight to sentences mentioning the reviewed business. As a smaller check, measure the error rate on test reviews that name a competitor ("instead of", "unlike", "compared to").

CNN (P-E1) — P10 (Confident false negative)

Label positive; predicted negative; $P(\text{positive})=0.00275886$. Exact excerpt: “Outstanding food quality and customer service by the waiter.”

Observed: BIAS (topic-word bias). The cause is not confirmed.. — this is a clearly positive review, and the model is almost certain it is negative. I can't see the reason from the text. My best guess is that "customer service" appears far more often in complaints than in praise in my 18k training reviews, and "waiter" also shows up mostly in complaint stories. "outstanding" may be rare in the sample.

Proposed validation check: occlusion test. Remove one word at a time from this review and record how $P(\text{pos})$ changes. If removing "customer service" makes the probability jump, the bias is confirmed. Then check the training vocabulary count for "outstanding": if it is rare, training on more data (100k+ reviews) should fix this kind of error.

CNN (P-E1) — P11 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.558424$. Exact excerpt: “Pendant des années, je travaillais dans le coin et, pendant des années, j'y ai mangé ...”

Observed: OOD (review written in French). — the review is in French. My vocabulary was built from English reviews, and accented characters are stored as escape codes in this dataset, so most tokens are unknown to the model. With almost no known words, the model has nothing to go on and outputs a probability close to 0.5. It missed the threshold by only 0.0016.

Proposed validation check: count non-English reviews in the test set (for example, reviews where more than 30% of tokens are unknown) and report their error rate separately. If it is close to 50%, either filter them out or mark the prediction as low-confidence. Success means the error rate on English reviews goes down once these are handled separately.

CNN (P-E1) — P12 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.553577$. Exact excerpt: “This is a refreshing change from your small quick dining options while in Vegas. You're ...”

Observed: HDG (hedged / mild sentiment), with MIX. — the reviewer praises the place by comparing it with bad alternatives, so many of the words are negative but describe *other* places ("stuck with", "run of the mill", "subpar"). The price comments ("marked up") add more negative words. The only clear positive is "refreshing change". The model ends up just under the threshold.

Proposed validation check: check calibration around the threshold. Apply temperature scaling (fit one temperature on the validation set) and see how many of the reviews within 0.05 of the threshold change class. Then tune a separate threshold for each length group and check whether near-threshold errors drop.

CNN (P-E1) — P13 (Near threshold)

Label negative; predicted positive; $P(\text{positive})=0.566825$. Exact excerpt: “I was here on a random Friday afternoon. It was okay, a different experience than I ...”

Observed: SAR (sarcasm), with HDG. — the reviewer is mildly negative ("okay", "won't come back", "unimpressive", "ran out") and uses sarcasm ("SO packed (read: not at all)"). There are also neutral-to-positive words ("perfect balance of crisp and gooey", "redeeming points", "decent liquor selection"). The tone is lukewarm rather than negative, so the probability sits just above the threshold.

Proposed validation check: build a small check set of test reviews containing sarcasm markers ("(read:", "yeah right", "SO" in capitals followed by a negation). Compare the CNN's error rate on this set with its overall rate. If it is much higher, report sarcasm as a separate failure mode. A quick test is whether keeping capital letters (no lowercasing) for all-caps words helps.

CNN (P-E1) — P14 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.552934$. Exact excerpt: “The Exchange has the biggest inventory of used CDs and movies in the city, so even if one ...”

Observed: HDG (hedged / mild sentiment), with CON. — the review is factual and mildly positive. The only evaluative parts are "biggest inventory", the price complaint, and "but otherwise it's all good". The "but" that

turns the review back to positive is removed by my stopword list. "doesn't have" adds a negation cue, so the model ends up just under the threshold.

Proposed validation check: the same "keep but" retrain as Error 3. Also check whether this review moves above the threshold after temperature scaling (see Error 12). Success means near-threshold errors on reviews containing "but otherwise" go away.

CNN (P-E1) — P15 (Near threshold)

Label positive; predicted negative; $P(\text{positive})=0.552112$. Exact excerpt: "Lovely place in the middle of nowhere. Bottom floor is the bar and the second floor is a ..."

Observed: HDG (hedged / mild sentiment), with BIAS. — "lovely" and "good" are positive, but "middle of nowhere" usually goes with complaints about location, and half the review is a neutral description of the floors. With only 15 tokens, a couple of words decide the result, and it lands just below the threshold.

Proposed validation check: short reviews are my CNN's weak slice (error rate 0.097). Test adding a kernel of width 2 for short phrases, and separately tune the threshold for reviews with 40 tokens or fewer on the validation set. Success means a lower error rate on the short slice.

CNN (P-E1) — P16 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.00349648$. Exact excerpt: "UPDATED. My initial very frustrated and dramatic review read as follows: Billing ..."

Observed: END (late sentiment shift), made worse by truncation. — the review quotes the reviewer's original angry review first ("fraudulent", "terrible billing practices", "not worth your time") and only then gives the positive update. The review has 200 tokens, but my model reads only the first 180. The cutoff falls right where the reviewer says "updating my review to 4 stars", so the clearest positive statement and everything after it ("they certainly get 5 for the way they managed this") are cut off. The model sees mostly the old negative review.

Proposed validation check: retrain with $\text{MAX_LEN} = 400$, or keep the first 90 and last 90 tokens instead of the first 180. Compare the error rate on the long slice (> 100 tokens; currently 0.092 for the CNN). Success means a lower error rate on long reviews, with this review classified correctly.

CNN (P-E1) — P17 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.00481228$. Exact excerpt: "I expected this location to be similar to the one in PHX: underwhelming and overpriced. ..."

Observed: NEG (negation / expectation cue misread), with CON. — the first sentence contains strong negative words ("underwhelming", "overpriced"), but they describe the reviewer's *expectation*, which the rest of the review contradicts. "However!!!" marks the turn and is kept, but the CNN doesn't model what "expected ... however" means. "I'm not totally sure" adds a negation cue. The positive ending ("much better", "deserves the hype") isn't enough.

Proposed validation check: compare CNN vs BiGRU on test reviews containing "expected" + "however" / "but". The BiGRU reads in order and should handle this pattern better. If it does, test a simple average of the CNN and BiGRU probabilities and check that the negation-slice error rate drops (currently 0.096).

CNN (P-E1) — P18 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.00715472$. Exact excerpt: "The food is crap. I'm not trying to be mean, but it really is horrible. I'd rather eat ..."

Observed: END (late sentiment shift). — almost the whole review is a strongly negative food complaint ("crap", "horrible", "worst nachos", "stomachs were so upset"). The rating is positive because the reviewer is really rating Thursday-night drinks, which they say only at the end ("Thursday night gets to keep the four star rating"). This review is 142 tokens, so it is not truncated. The model sees the ending, but the negative part is much larger.

Proposed validation check: give the last sentences more weight. Train a variant that adds a second input made of only the last 40 tokens, and combine the two scores. Compare error rates on test reviews whose last

sentence has the opposite sentiment from the rest. As a simpler first check, the star-rating feature from Error 7 would also catch "four star rating".

CNN (P-E1) — P19 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0100914$. Exact excerpt: "A friend and I visited this theatre the other day and the movie we intended on seeing was ..."

Observed: MIX (mixed sentiment), with BIAS. — the review starts with a negative event ("no longer showing", "cancelled" twice, "apologized"), and those words are common in complaints. The positive verdict is clear to a human ("extremely friendly", "very impressed", "will drive out of my way to come ... again"), but "customer service" again pulls towards negative, as in Error 10.

Proposed validation check: the same occlusion test as Error 10. Remove "customer service" and "cancelled" one at a time and record how $P(\text{pos})$ changes. If both make the probability rise sharply, these topic words are biasing the model. Then retrain on a larger sample (100k reviews) and check whether reviews containing "customer service" still have a higher false-negative rate than average.

CNN (P-E1) — P20 (Long-review / negation slice)

Label positive; predicted negative; $P(\text{positive})=0.0110541$. Exact excerpt: "It's hard to find a non corporate coffee outlet in Las Vegas, but I found this place on ..."

Observed: NEG (negation cue misread). — the review has several negation and negative-looking words that aren't negative here: "hard to find", "non corporate", "not just a conversation piece", "No pastries offered". I kept "not" and "no" on purpose, because they usually flip sentiment, but in this review they don't attach to anything negative. The positives ("good coffee", "much better", "good choice") are short and get outweighed.

Proposed validation check: count negation words only when they come right before an adjective or sentiment word (for example, "not good" counts but "not just" doesn't). Retrain with a simple negation-scope rule that joins "not" + the next word into one token (not_good), and compare the negation-slice error rate (currently 0.096) and the overall macro-F1.

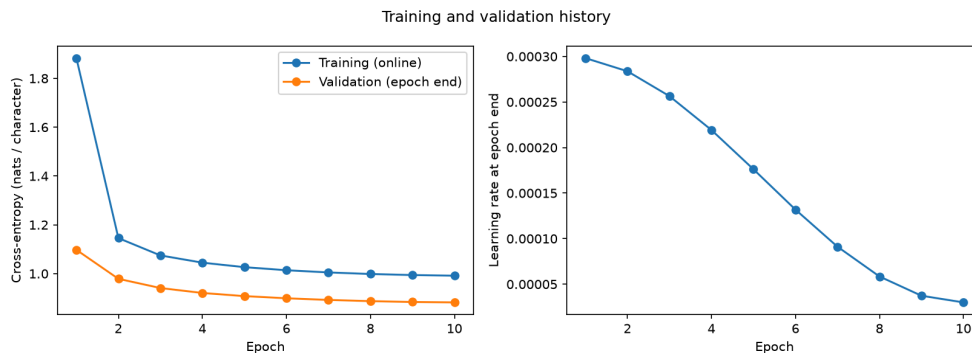
Appendix B Additional Figures and Audit Evidence

Both members' saved curves, fixed visual examples, completed 30-case audits, and Kaggle Public-score screenshots are documented below.

Figure Record

B1 Yuyao Task 1 loss curves

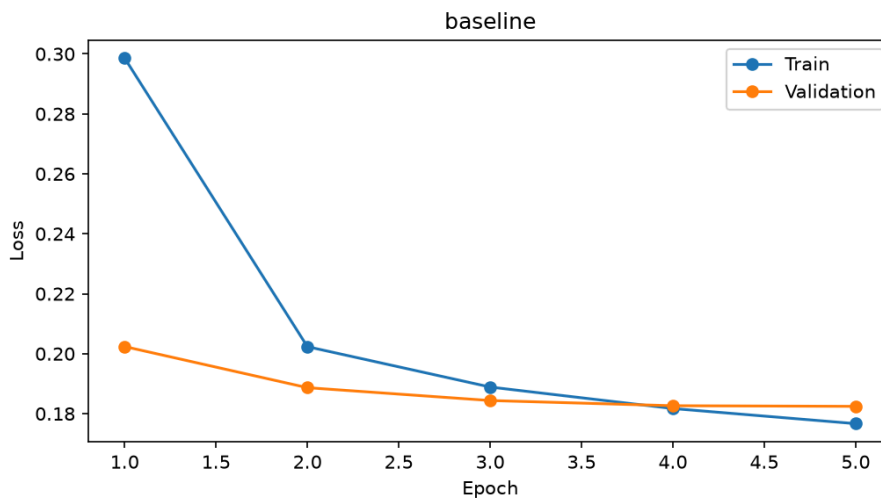
Yuyao, train_20260927T000510Z_b9df2210



Training and validation cross-entropy by epoch for the sampled TinyStories data; matched-checkpoint metrics are in Section 2.3.

task1_llm/Yuyao_Ding/outputs/train_20260927T000510Z_b9df2210/loss_curves.png

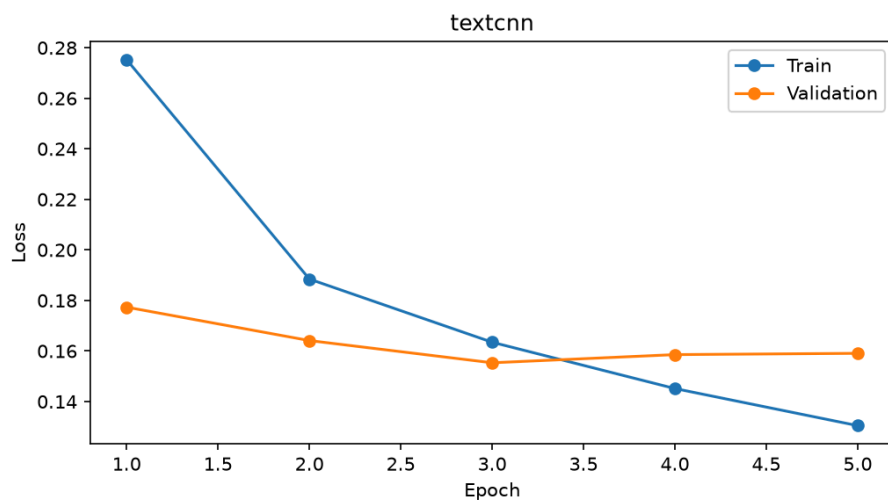
B2 Yuyao Task 2 baseline loss



Train/validation loss by epoch; selected epoch 5. Source:

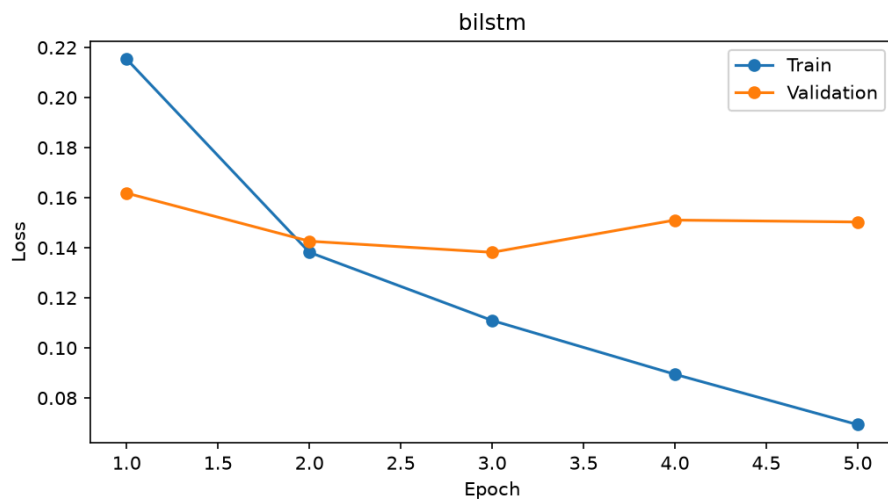
task2_sentiment/Yuyao_Ding/outputs/task2_formal_20260927T020054Z_ba87cd57/baseline/loss_curve.png

B3 Yuyao Task 2 TextCNN loss



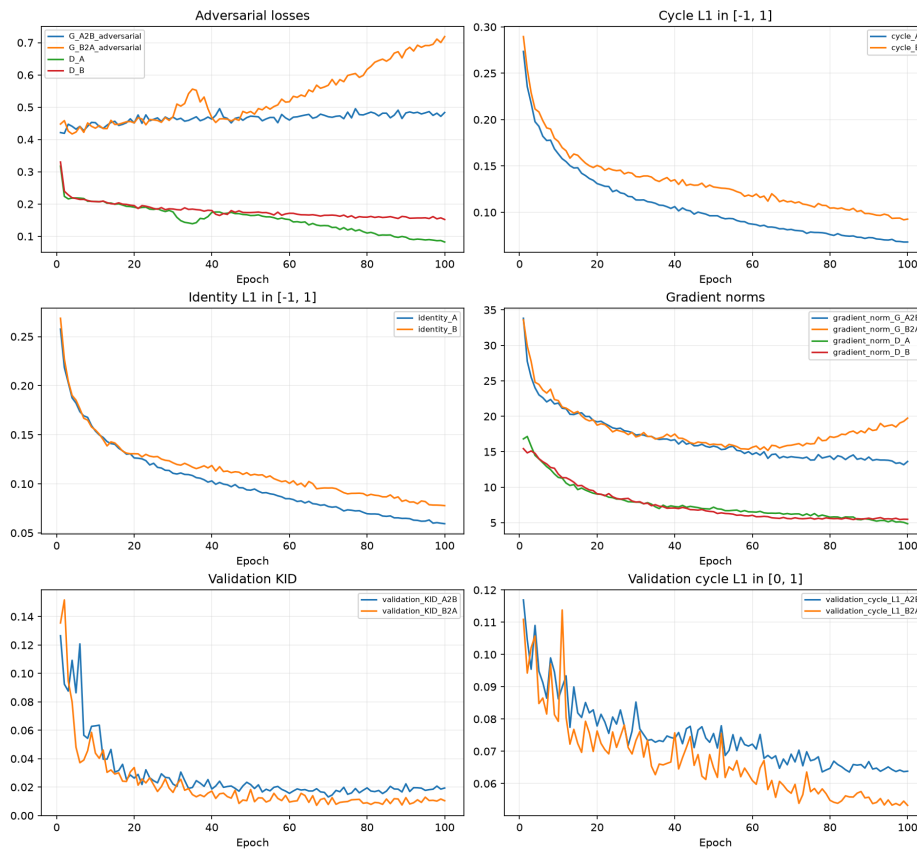
Train/validation loss by epoch; selected epoch 3. Source: task2_sentiment/Yuyao_Ding/outputs/task2_formal_20260927T020054Z_ba87cd57/textcnn/loss_curve.png

B4 Yuyao Task 2 BiLSTM loss



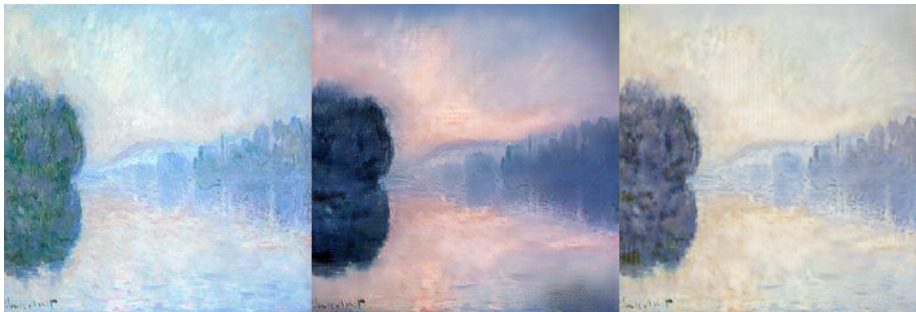
Train/validation loss by epoch; selected epoch 3. Source: task2_sentiment/Yuyao_Ding/outputs/task2_formal_20260927T020054Z_ba87cd57/bilstm/loss_curve.png

B5 Yuyao Task 3 training curves



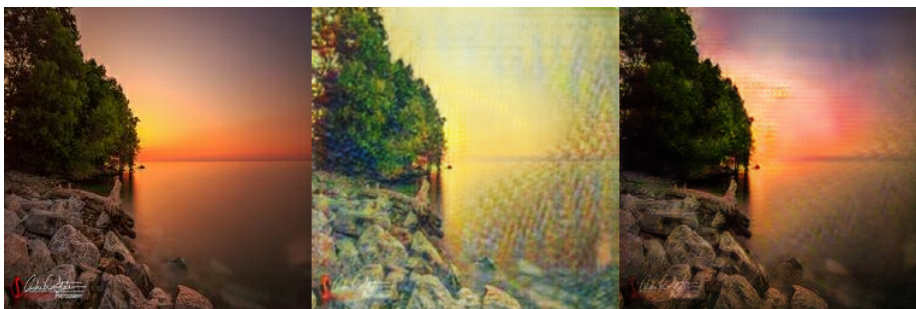
Generator, discriminator, cycle, and identity losses over 100,000 updates. Source: [task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/training_curves.png](https://github.com/YuyaoDing/task3_gan/blob/master/outputs/task3_formal_20260927T075834Z_d335ae80/training_curves.png)

B6 Yuyao Task 3 A → B fixed panel



Monet river: input, translation, and cycle reconstruction, left to right. Source: [task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/A2B_0260d15306_input_translation_cycle.png](https://github.com/YuyaoDing/task3_gan/blob/master/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/A2B_0260d15306_input_translation_cycle.png)

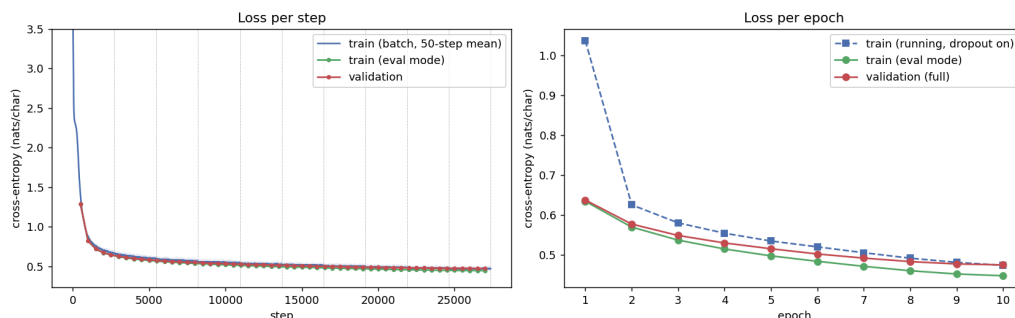
B7 Yuyao Task 3 B → A fixed panel



Sunset shoreline: input, translation, and cycle reconstruction, left to right. Source: [task3_gan/Yuyao_Ding/outputs/task3_formal_20260927T075834Z_d335ae80/evaluation_20260927T130841Z_a267de/heldout_test/B2A_02620a36f7_input_translation_cycle.png](#)

B8 Pratiksha Task 1 loss curves

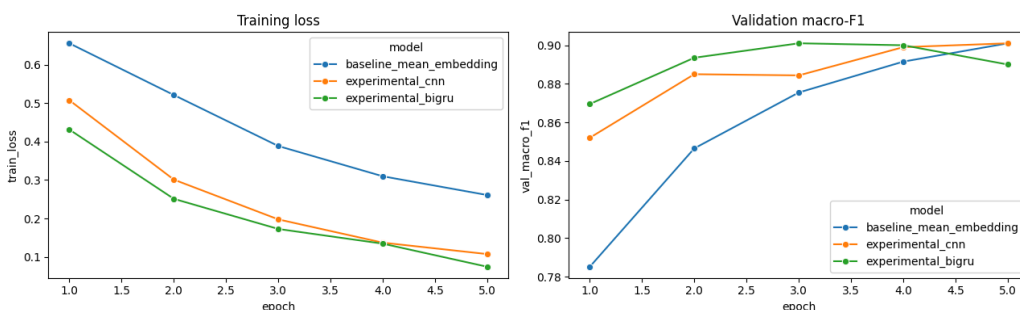
Pratiksha, 20261005-223946_L8H8C512T512



Training and validation cross-entropy over 10 epochs. Source: [task1_llm/Pratiksha_Kaushik/outputs/plots/loss_curves.png](#)

B9 Pratiksha Task 2 training curves

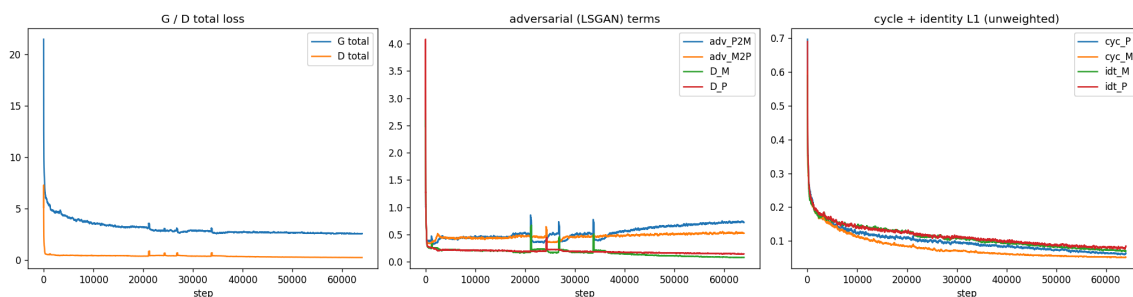
Pratiksha, all-model run of 2026-10-05



Training loss and validation macro-F1 per epoch for P-B, P-E1 and P-E2. Source: [task2_sentiment/Pratiksha_Kaushik/outputs/training_loss_and_val_f1.png](#)

B10 Pratiksha Task 3 training curves

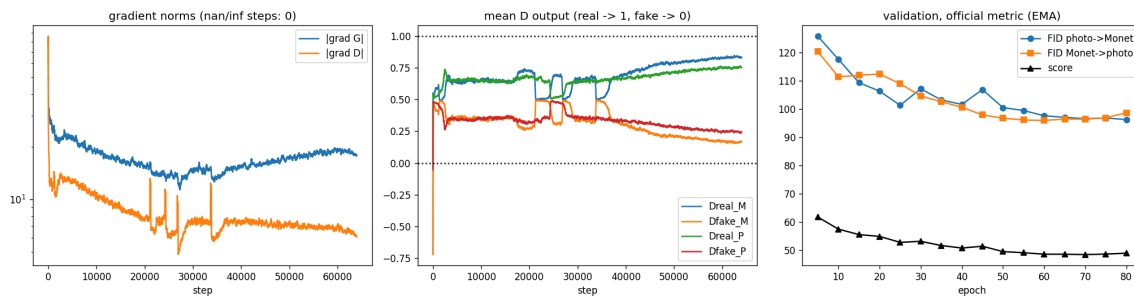
Pratiksha, CycleGAN v3 final run



Generator, discriminator, adversarial, cycle and identity losses over 64,000 steps. Source: [task3_gan/Pratiksha_Kaushik/outputs/figures/loss_curves.png](#)

B11 Pratiksha Task 3 stability curves

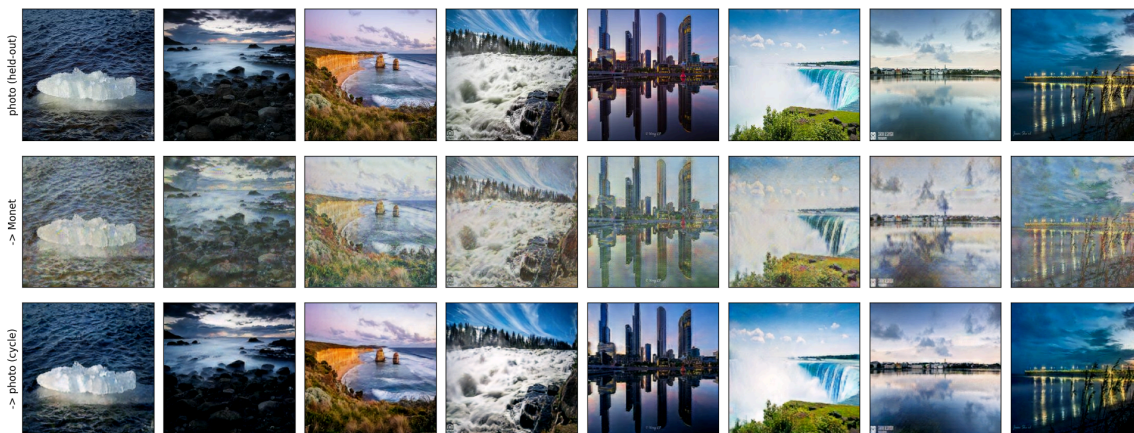
Pratiksha, CycleGAN v3 final run



Gradient norms (0 NaN/Inf steps), mean discriminator outputs, and validation score per checkpoint. Source: task3_gan/Pratiksha_Kaushik/outputs/figures/stability_curves.png

B12 Pratiksha Task 3 photo → Monet

Pratiksha, v3 final model



Held-out photos (top), translations (middle) and cycle reconstructions (bottom). Source: task3_gan/Pratiksha_Kaushik/outputs/figures/qual_photo2monet.png

B13 Pratiksha Task 3 Monet → photo

Pratiksha, v3 final model



Monets (top), translations (middle) and cycle reconstructions (bottom). Source: task3_gan/Pratiksha_Kaushik/outputs/figures/qual_monet2photo.png

Human Audit Attachments

Yuyao: 30 fixed anonymous cases, two completed rater sheets, instructions, and input/translation image pairs are in the audit folder. Section 4.6 gives the pooled scores and inter-rater agreement; audit_results.json preserves the aggregate evidence.

Pratiksha: 30 fixed anonymous cases (images/), two completed rater sheets (rater1.csv, rater2.csv), the hidden direction key (_key.csv), and the computed summary (audit_summary.csv, audit_scores_per_sample.csv) are in task3_gan/Pratiksha_Kaushik/outputs/human_audit/. Section 4.6 gives the pooled scores and inter-rater agreement.

Kaggle Attachments

Yuyao: task3_gan/Yuyao_Ding/outputs/kaggle_public_submissions.png shows her submission.csv Public score of -52.8347. An individual rank and Private score were not shown.

Pratiksha: task3_gan/Pratiksha_Kaushik/outputs/kaggle/kaggle_submission_final.png shows her final submission (uploaded as submission.csv) with a Public score of -50.2351; kaggle_submission_earlier.png shows the earlier submission_official_v3.csv at -50.2367. A Private score was not shown.

Q Search

DATA 266 Fall 2026 - GAN Image Style Transfer

Submit Prediction

...

OverviewDataCodeModelsDiscussionLeaderboardRulesTeamSubmissions

Automatically select from your best scoring submissions. [Learn more](#)

Auto-selection candidates ?

AllSuccessfulSelectedErrors

Recent ▼

Submission and Description

Score ⌚Select

✓

submission.csv

Complete · Pratiksha Kaushik · 41s ago

-50.2351

✓

DATA 266 Fall 2026 - GAN Image Style Transfer

Unpaired Image-to-Image Translation using Generative Adversarial Network (GAN)



OverviewDataCodeModelsDiscussionLeaderboardRulesTeamSubmissions

Leaderboard

Raw Data

Refresh

YOUR RECENT SUBMISSION



submission_official_v3.csv

Submitted by Pratiksha Kaushik · Submitted 4 days ago

Score: -50.2367

Jump to your leaderboard position

36

PairProgramming_Team_41



-50.2351

4

11h



New personal best! Your latest submission scored -50.2351, improving on your previous best of -50.2367.

[Tweet this](#)

Submission and Description

Score ⓘ

Select

**submission_official_v3.csv**

Complete · Pratiksha Kaushik · 4d ago

-50.2367**submission.csv**

Complete · Yuyao Ding · 4d ago

-52.8347**submission_official_v1.csv**

Complete · Pratiksha Kaushik · 4d ago

-51.3334